

東吳大學巨量資料管理學院金融科技創新與應用

School of Big Data Management, Soochow University

FinTech Innovation and Application Program

三因子結合機器學習:市場投資組合的智慧建構

Integrating Three Factors with Machine Learning:

Smart Construction of Market Investment Portfolios

何書維

Shu-Wei Ho

指導教授: 鄭宏文

Advisor: Hung-Wen Cheng

中華民國 114 年 4 月

摘要

本研究結合 Fama-French 三因子——規模 (Size)、帳面市值比 (B/M) 與動能 (Momentum) ——與機器學習技術，透過預測各因子的資訊係數 (IC)，動態調整權重，以智慧化方式建構臺灣股票投資組合。資料採自 TEJ，涵蓋 1999 年 1 月至 2025 年 2 月之上市櫃月度資料，並僅保留 2025 年 1 月仍存續之普通股。研究比較線性迴歸 (OLS)、隨機森林 (RF) 與多層類神經網路 (NN1 - NN5) 三大模型族群。評估指標包括累積報酬、勝率、夏普比率、樣本外 R^2 與再投資累積報酬。

實證結果顯示，非線性模型優勢明顯：RF 在所有持股等分 (10-964 檔) 皆長期打敗大盤，勝率 0.54-0.63、夏普比率 0.25-0.44，且樣本外 R^2 為正 (0.045)；深層 NN3-NN4 於中等持股數 (10-192 檔) 亦展現不錯的勝率 (≈ 0.55) 與夏普 (0.20 以上)，但在極度分散時表現略遜。相對地，OLS 與淺層 NN1、NN2 因缺乏非線性捕捉能力，勝率僅近隨機水準，樣本外 R^2 為負，且再投資報酬曲線自 2020 年後顯著回吐。

綜合而言，本研究證實「因子投資 $\times$ AI 預測」之可行性：利用機器學習動態配置因子權重，可顯著提升長期勝率與風險調整後報酬，並穩定超越臺灣加權股價指數。RF 在可解釋性、穩健性與全周期表現上最具優勢；深層 NN 為中等持股數的進階選擇；而線性模型僅適合作為基準對照。此一結論為資產管理領域導入資料驅動策略提供具體指引，亦提醒投資者在實務上須兼顧模型容量、流動性與執行成本。

關鍵字： 機器學習、資訊係數、帳面市值比、規模、動能、類神經網路、／ 隨機森林、線性迴歸

Abstract

This study integrates the Fama–French three factors—Size, Book-to-Market (B/M), and Momentum—with machine-learning techniques. By forecasting each factor’s Information Coefficient (IC) and dynamically adjusting their weights, it intelligently constructs equity portfolios for the Taiwanese market. The dataset, sourced from TEJ, covers monthly observations from January 1999 to February 2025 and retains only common stocks still listed as of January 2025. Three model families are compared: Ordinary-Least-Squares regression (OLS), Random Forests (RF), and multi-layer neural networks (NN1–NN5). Performance is evaluated using cumulative return, hit ratio, Sharpe ratio, out-of-sample R^2 , and reinvested cumulative return.

Empirical results reveal a clear advantage for non-linear models. RF outperforms the market across all portfolio sizes (10–964 stocks), achieving hit ratios of 0.54–0.63, Sharpe ratios of 0.25–0.44, and a positive out-of-sample R^2 (0.045). Deep networks NN3–NN4 also deliver solid performance for moderate portfolio sizes (10–192 stocks), with hit ratios around 0.55 and Sharpe ratios above 0.20, though they lag slightly when portfolios are extremely diversified. In contrast, OLS and shallow networks NN1–NN2, which lack the capacity to capture non-linearity, show near-random hit ratios, negative out-of-sample R^2 values, and pronounced drawdowns in reinvested returns after 2020.

Overall, the study confirms the viability of “factor investing × AI forecasting.” Dynamically allocating factor weights with machine learning can markedly improve long-term hit ratios and risk-adjusted returns, consistently beating the Taiwan Stock Exchange Capitalization Weighted Stock Index (TAIEX). RF strikes the best balance of interpretability, robustness, and full-cycle performance; deep neural networks are an advanced option for moderately sized portfolios; and linear models serve mainly as benchmarks. The findings offer concrete guidance for data-driven asset-management strategies while reminding investors to consider model capacity, liquidity, and execution costs in practice.

Keyword : Machine Learning 、 IC 、 book-to-market 、 size 、 momentum 、 Neural Network 、 Random Forest 、 Linear Regression

目錄

一、緒論.....	2
1.1 研究背景與動機	2
1.2 研究目的.....	2
二、文獻探討	3
2.1 傳統因子模型與機器學習的整合應用	3
2.2 深度學習在資產定價中的應用	3
三、研究方法	4
3.1 資料集說明	4
3.2 股票因子計算與資料前處理	5
3.3 模型訓練.....	6
3.4 評估方法.....	9
四、研究成果	11
4.1 累積報酬分析	11
4.2 勝率分析.....	15
4.3 夏普比率分析	16
4.4 樣本外 R-square 分析.....	17
4.5 再投資累積報酬率分析	17
五、研究結論	22
六、心得感想	22
七、參考文獻	23
八、附錄.....	23

一、緒論

1.1 研究背景與動機

在現代金融市場中，資產定價模型的發展一直是學術界與實務界關注的焦點。自從 Sharpe[1]提出資本資產定價模型（CAPM）以來，Fama 與 French[2]進一步發展了三因子模型，將公司規模（Size）與帳面市值比（Book-to-Market, B/M）納入考量，顯著提升了對股票報酬率橫斷面差異的解釋力。後續 Carhart[3]於 1997 年加入動能（Momentum）因子，形成四因子模型，更全面捕捉市場異常現象。儘管這些因子模型在不同市場中廣泛驗證，但因其假設線性關係及參數靜態不變，仍無法有效應對市場結構日益複雜且快速變化的挑戰。

近年來，隨著大數據與人工智慧技術的蓬勃發展，機器學習方法在金融資料分析中展現出強大的非線性建模與預測能力。特別是在資產管理領域，線性迴歸（OLS）、隨機森林（RF）與類神經網路（NN）等方法，能夠自動從海量資料中提取重要特徵，有效提升預測準確性。根據 Gu、Kelly 以及 Xiu[4]於 2020 年時的研究，將機器學習技術應用於資產定價，有助於發掘傳統方法無法捕捉的深層次結構關係，並提高模型表現的穩健性與適應性。

因此，本研究以規模、動能及帳面市值比三因子為基礎，結合機器學習技術，藉由預測各因子的資訊係數（IC 值）進行動態因子權重配置，進而建構智慧型市場投資組合。研究動機著重於藉助資料驅動的方法，不僅提升投資決策的精確度，並希望能在長期投資中提高勝率、增強報酬的穩定性，以及實現超越大盤（台灣加權股價指數，TWII）表現的目標。在金融市場高度競爭與波動的環境下，如何結合因子投資策略與 AI 預測技術，已成為資產管理領域亟需探索的重要課題，本研究即以此為核心出發點。

1.2 研究目的

本研究旨在以台灣上市櫃公司之月度資料為基礎，運用機器學習技術（線性迴歸、隨機森林與類神經網路）預測帳面市值比、規模及動能三個因子的未來資訊係數（IC 值），並根據預測結果進行因子權重動態調整，建構不同等份排序的市場投資組合。透過模擬各種投資組合的報酬表現，與台灣加權股價指數進行比較，實證檢視所建構之智慧型投資策略在提高勝率、提升投資穩定性及長期超越大盤報酬率方面的可行性與成效。

二、文獻探討

2.1 傳統因子模型與機器學習的整合應用

傳統資產定價模型，如 Fama 與 French[2]提出的三因子模型，透過納入市場風險、市值規模（Size）與帳面市值比（Book-to-Market, B/M）三個因子，大幅提升了對資產橫斷面報酬差異的解釋力，並在不同市場環境中被廣泛驗證。然而，這類模型多假設因子與資產報酬之間存在線性且靜態的關係，無法充分捕捉現代金融市場中日益複雜、非線性且高度動態的特性。Gu、Kelly 與 Xiu[5]指出，傳統線性方法在面對高維度且非線性互動的市場資料時，預測力存在明顯不足；相較之下，機器學習技術能夠藉由資料驅動的方式，自動從大量特徵中提取非線性結構，並顯著提升樣本外的預測表現。

進一步地，動能（Momentum）因子作為資產市場中重要的異常現象，早期由 Jegadeesh 與 Titman[6]實證發現，過去表現優異（劣勢）的股票在未來一段期間內仍有可能持續優異（劣勢）表現，形成可獲取超額報酬的投資策略。Bui、Kong 等人[7]與針對台灣股市進行研究，發現透過機器學習模型（如神經網路、部分最小平方方法）建立的多空動能策略，不僅能顯著提高投資組合的勝率與報酬，且能有效捕捉傳統方法難以辨識的動能信號。這些研究結果指出，機器學習技術能夠在動能因子建模中挖掘出更高層次的非線性關聯性，並提升策略的穩健性與適應性，為智慧型投資組合的建構提供了重要依據。

2.2 深度學習在資產定價中的應用

深度學習（Deep Learning）作為機器學習的重要分支，展現出處理大量、結構複雜且高度非線性資料的卓越能力。Feng、He 與 Xiu[8]首度系統性地將深度學習模型應用於橫斷面資產定價問題，利用多層感知器（Multilayer Perceptron, MLP）結合多因子資料進行預測。研究結果顯示，深度學習模型在捕捉因子間的非線性結構方面，優於傳統線性回歸及其他機器學習方法，且在樣本外的預測表現顯著提升。

進一步地，Sirignano 與 Cont[9]針對金融市場中的價格形成機制進行研究，運用遞歸神經網路（Recurrent Neural Networks, RNN）等深度學習架構，揭示了金融市場價格變動中存在的普遍非線性特徵。他們的實證結果指出，深度學習技術能有效捕捉時間序列資料中複雜的跨期依賴關係，並能提升對金融資產價格動態變化的預測能力。

這些研究成果共同證實，深度學習方法在資產定價領域具有極高潛力，不僅能

處理傳統模型難以捕捉的高維非線性特徵，也為建構更智慧、具適應性的市場投資組合策略提供了堅實的理論與實證基礎。

三、研究方法

本研究以規模（Size）、動能（Momentum）及帳面市值比（Book-to-Market, B/M）三因子為基礎，結合機器學習技術，透過預測各因子的資訊係數（Information Coefficient, IC）進行動態因子權重配置，進而建構智慧型市場投資組合。研究流程如圖 1 所示。

整體研究設計可分為二個步驟，分別為：「股票因子的計算」以及「機器學習模型訓練與績效分析」。以下將針對各步驟進行詳細說明。

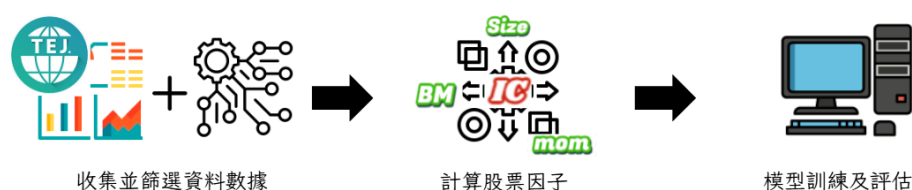


圖 1、研究流程圖

3.1 資料集說明

本研究所使用之資料來源為台灣經濟新報（Taiwan Economic Journal, TEJ）資料庫，選取其股價月資料作為主要分析對象，涵蓋期間自 1999 年 1 月至 2025 年 2 月，資料橫跨 25 年。本研究僅納入 2025 年 1 月時仍存續之上市及上櫃普通股，並以股票代碼作為公司識別基礎。TEJ 資料庫專門蒐集與台灣及華人地區企業相關之各類金融與經濟資料，內容涵蓋上市、上櫃及興櫃公司之基本資料、財務報表、股價與交易資訊、法人持股情形、公司治理評比結果及企業社會責任（CSR）報告等。其資料主要來源包括台灣證券交易所、證券櫃檯買賣中心、金融監督管理委員會、公開資訊觀測站以及各企業自行揭露之官方資訊。TEJ 資料庫以其資料之完整性與高度可信度，廣泛應用於學術研究與實務分析。因此，本研究選擇 TEJ 作為主要資料來源，以確保資料品質與研究結果之可靠性與嚴謹性。

3.2 股票因子計算與資料前處理

根據第 1.2 節所述，本研究運用機器學習技術（包括線性迴歸、隨機森林與類神經網路）以預測帳面市值比因子、規模因子及動能因子之未來資訊係數（Information Coefficient, IC）表現。以下將針對上述三個因子的公式與定義進行詳細說明。

3.2.1 帳面市值比(Book-to-Market, BM)

如公式 1 所示，帳面市值比亦稱淨值市價比，指將公司之帳面價值（股東權益）與其市值進行比較，用以衡量企業相對於市場評價之內在價值。一般而言，帳面市值比可作為評估公司是否被高估或低估的指標，數值較高者可能代表公司價值被低估，反之則可能顯示市場對其前景較為悲觀。

$$\text{帳面市值比} = \frac{\text{普通股股東權益}}{\text{市值}} = \frac{1}{\text{股價淨值比}} \quad (1)$$

3.2.2 規模(Size)

通常以公司市值（Market Capitalization）作為衡量基準，用以反映公司規模對股票報酬率之影響。一般而言，市值較小的公司（小型股）因承擔較高的風險，常被市場要求較高的預期報酬；反之，市值較大的公司（大型股）則風險較低，預期報酬亦相對較低。因此，規模因子常被用於捕捉小型股相對大型股的超額報酬現象，並廣泛應用於資產定價模型與投資組合建構之中。其市值及規模的公式如公式 2 及公式 3 所示。

$$\text{市值} = \text{收盤價} * \text{流通在外的股數} \quad (2)$$

$$\text{規模} = \log(\text{市值}) \quad (3)$$

3.2.3 動能(Momentum)

動能因子如公式 4 所示，意指的是資產價格延續過去趨勢的一種現象，具體表現為過去表現良好的資產在未來一段期間內傾向持續上漲，而過去表現不佳的資產則傾向持續下跌。這種現象在實證金融研究中被廣泛觀察到，因此動能因子常被納入資產定價模型，用來解釋市場報酬中的系統性變異。在這裡，本研究將採用形成期 12 個月計算以報酬率形成的動能，概念為 $t-1$ 期股票收盤價除以 $t-12$ 期股票收盤價取對數，不受基期和時間影響

$$\text{動能} = \ln\left(\frac{S_{t-1}}{S_{t-12}}\right) \quad (4)$$

由於將上面描述的三因子(bm、size、mom)計算之後，會發現部分公司的特定年分會出現缺值的情況，針對這一部分，我們將含有缺值的年份，將其因子的值以及下個月月報酬部分各以做中位數的計算，並將其算出來的值填補在缺值的位置。

3.2.4 資訊係數(Information Coefficient, IC)

所謂的資訊係數(IC)，即為股票因子與股票下棋報酬率的相關係數，其中，常見 IC 值有二種算法 Normal IC 及 Rank IC，分別如公式 5、6 所示：

➤ Normal IC

$$\text{Normal } IC_t = \text{corr}(X_t, R_{\{t+h\}}) \quad (5)$$

➤ Rank IC

$$\text{Rank } IC_t = \text{corr}(r(X_t), r(R_{\{t+h\}})) \quad (6)$$

所謂 Normal IC，指在時間點 t 時因子與持有 h 個月的報酬率間的相關係數，而 Rank IC 相對於前者，差別在於會先將因子進行排序，在計算在時間點 t 時因子與持有 h 個月的報酬率間的相關係數，其中 $r(X_t)$ 和 $r(R_{\{t+h\}})$ 表示已先將動能因子及對數報酬率進行 Rank 排序，再計算相關係數。

IC 具有以下五點特性：

1. 透過 IC 值可以判斷因子對下期報酬率的預測能力
2. IC 值介於 -1 至 1 之間
3. 當 IC 值 > 0 時，表示當期因子與下期報酬率有正向關係
4. 當 IC 值 < 0 時，表示當期因子與下期報酬率有負相關。
5. IC 值之絕對值越大，表示該因子對下期報酬率具有較大影響力

3.3 模型訓練

本研究選擇三種不同的機器學習模型：線性回歸模型(Linear Regression)、隨機森林(Random Forest)以及類神經網路(Neural Network)來訓練三因子建置投資組合，評估是否能夠有效地提高勝率、投資穩定性及長期超越大盤報酬率

3.3.1 線性回歸模型(Linear Regression)

線性回歸模型 (Linear Regression Model) 是一種基本的統計建模方法，如公式 7 所示，該模型用來描述一個因變數（目標變數）與一個或多個自變數（解釋變數）之間的線性關係。目的是找出最適合數據的直線或超平面，以便進行預測或推論。

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon \quad (7)$$

其中：

Y：模型的輸出，亦即預測或解釋的目標變數，在此研究中為因子最後建立投資組合的決策值

β_0 ：截距，反映 Y 的基礎水準

$\beta_1, \beta_2, \dots, \beta_p$ ：各個自變數對應的權重

$X_1, X_2, \dots, X_p$ ：自變數，意即輸入的因子特徵

ϵ ：隨機誤差，表示模型無法解釋的部分

3.3.2 隨機森林(Random Forest)

隨機森林 (Random Forest) 為一種基於集成學習 (Ensemble Learning) 概念之監督式學習方法，主要應用於分類 (Classification) 與回歸 (Regression) 問題。該方法由 Breiman (2001) 提出，其核心思想為結合多棵具有高變異性之決策樹 (Decision Trees)，透過投票 (voting) 或平均 (averaging) 機制以提升模型整體之穩健性與預測精度，同時有效降低單一模型可能面臨之過擬合 (Overfitting) 問題。

在隨機森林的訓練過程中，首先對原始資料集進行多次自助抽樣 (Bootstrap Sampling)，即從原資料集中隨機有放回地抽取樣本以生成多個訓練子集。每個子集將用以建立一棵決策樹；在建構決策樹的每一個節點時，隨機選擇部分特徵進行最佳分裂 (Best Split)，而非考量全部特徵，此舉進一步增加了模型之多樣性 (Diversity)。

隨機森林模型之預測機制，根據任務類型而有所不同：於分類問題中，取各棵樹所預測類別之眾數 (Majority Voting) 作為最終預測結果；於回歸問題中，則取各棵樹預測值之算術平均 (Arithmetic Mean) 作為最終預測值。

形式化而言，假設建立了 B 棵決策樹，記第 b 棵樹為 $T_{b(x)}$ ，則：

- 針對分類問題，隨機森林之預測結果如公式 8 所示：

$$\hat{y} = \text{majority_vote}\{T_{b(x)}\}_{b=1}^B \quad (8)$$

- 而針對回歸問題，則如公式 9 所示：

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_{b(x)} \quad (9)$$

其中， x 表示輸入特徵向量， $\hat{y}$ 為隨機森林之最終預測結果。

本研究將會以 `random_state = 1042` 去做訓練，並分析與其他模型輸出的結果。

3.3.3 類神經網路(Neural Network)

類神經網路 (Neural Network, NN) 是一種模仿人腦神經元連結與訊息傳遞機制的數值計算架構，核心由大量節點 (神經元) 與權重化連線組成，通常分為輸入層、若干隱藏層以及輸出層。每個神經元會對前一層輸出進行加權求和，然後通過非線性活化函數 (如 ReLU、Sigmoid 或 Tanh) 轉換，以捕捉資料中的複雜關係。訓練過程中，網路透過前向傳播 (forward pass) 產生預測值，再以損失函數 (loss function) 計算預測與真值之間的誤差，最後運用反向傳播 (back-propagation) 結合梯度下降或其變種 (如 Adam、RMSprop) 來調整權重，逐步最小化誤差。

隨著計算能力與資料量的提升，NN 發展出各種專門結構：捲積神經網路 (CNN) 善於從影像中自動萃取空間特徵，廣泛用於人臉辨識、自動駕駛與醫療影像診斷；循環神經網路 (RNN) 及其改良版 LSTM、GRU 能夠保留序列記憶，適合語音辨識、機器翻譯與時間序列預測；而 Transformer 架構藉由自注意力機制 (self-attention) 並行處理序列，已成為大型語言模型與生成式 AI 的主流技術。此外，生成對抗網路 (GAN) 與變分自編碼器 (VAE) 則能合成高擬真的圖像、音樂與文本。

儘管類神經網路在多項任務中超越傳統演算法，仍面臨可解釋性不足、訓練資料需求龐大、對對抗樣本脆弱以及能源消耗高等挑戰。為了提升可靠性與效率，研究者積極探索如少樣本學習 (few-shot learning)、自監督學習 (self-supervised learning)、神經網路壓縮與硬體加速等方向。總體而言，類神經網路以其靈活且高度可擴充的結構，已成為推動現代人工智慧迅速演進的關鍵動力。

在本研究中，我們將利用多個不同維度的線性層來構建我們的類神經網路，其架構如圖 2 所示：

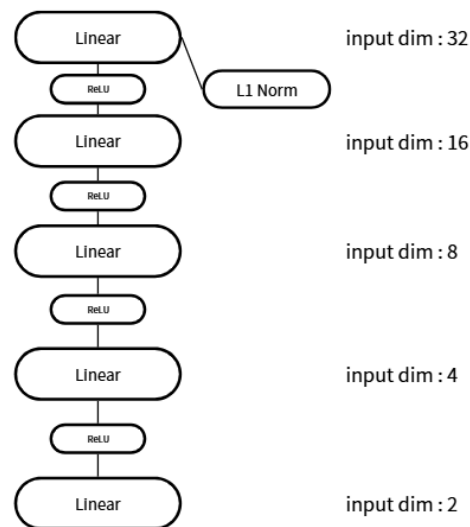


圖 2、類神經網路架構

其中，第一層線性層會加入 L1 Normalization 來降低出現 overfitting 的可能性。其加入的權重為 $1e-4$ 。此外，本研究會將架構由下至上拆分成五個類神經網路去訓練評估，每次的訓練皆以 random seed = 1042、learning rate = $1e-4$ 、epochs = 100 及 batch size = 32 去進行，並以 Mean Square Error (MSE) 計算 loss 來評估模型。

3.4 評估方法

本研究將以五種評估方法來分析利用三種不同機器學習模型對三因子預測，並篩選出最佳的投資組合，此種五種評估方法分別為累積報酬、勝率、夏普比率、樣本外的 R-square 以及再投資累積報酬，以下將會詳細說明：

3.4.1 累積報酬(Cumulative Return)

累積報酬是衡量投資自起始日期至某一特定時間點之整體盈虧的指標，反映資產價格變動與任何再投資收益（如股利、利息）的總合效果。計算方式通常以期末價值減去期初價值，再除以期初價值，或用連續期間的單期報酬率相乘後減一，因而能直觀展示「若當初投入一筆資金並持續至今，其價值成長多少」。累積報酬有助於投資人比較不同期間或策略的績效，評估長期投資成果及市場波動對整體財富的影響。

3.4.2 勝率(Win Rate)

在金融中，勝率通常指的是一項投資策略或交易系統在所有已完成交易中，獲利筆數占總筆數的比例；它揭示的是「多數時候能否賺錢」而非「最終能否賺大錢」。高勝率雖代表較常出現小額獲利，但若每次獲利幅度遠小於偶爾出現的虧損幅度，依然可能整體虧損；反之，低勝率策略只要單筆獲利足以覆蓋多次小虧，也能累積正報酬。因此，評估勝率時必須同時考量盈虧比（平均獲利／平均虧損）、風險控制與資金配置，才能真正衡量策略的長期可行性，而非單純被勝率數字所迷惑。

3.4.3 夏普比率(Sharpe ratio)

夏普比率是衡量投資組合「單位風險所獲報酬」的指標，計算方式如公式 10

$$\text{Sharpe Ratio} = (R_p - R_f) / \sigma_p \quad (10)$$

其中 R_p 為投資組合的年化報酬率、 R_f 為無風險利率（常以短期公債殖利率近似），而 σ_p 則代表投資組合報酬率的年化標準差。比率愈高，表示在考量波動風險後，該投資組合所帶來的超額報酬愈優異；若 Sharpe 比率為負，則意味著在同樣的風險水準下，投資組合表現甚至低於無風險資產。投資人通常以此來比較不同資產或基金的風險調整後績效，並作為優化資產配置與風控策略的重要參考

3.4.4 樣本外的 R-square

樣本外 R-square（又稱預測 R^2 ）是以未參與模型估計的保留資料來衡量模型預測力的指標，其計算方式為比較模型在樣本外的殘差平方和與僅用目標變數平均值作為預測時的殘差平方和：若結果接近 1，表示模型在新資料上的解釋能力仍然強；若為 0，代表模型與簡單的平均值預測表現相當；若為負值，則意味著模型在樣本外甚至比單純用平均值還差。由於它不受「過度擬合」所造成的樂觀偏誤影響，樣本外 R-square 能更真實反映模型面對未知資料時的預測效能，因而常被用來比較不同模型或在交叉驗證、滾動預測等情境中評估模型的實用性。

3.4.5 再投資累積報酬

如公式 11 所示，再投資累積報酬指的是將每一期獲得的利息、股息或其他收益立即投入原先的投資標的或相同報酬率的資產中，使得本金隨時間持續放大，進而產生「利滾利」效果；隨著期間愈長、報酬率愈高或再投資頻率愈密集，複利力量便愈顯著，最終累積的總報酬呈現指數型增長，而不僅是簡單的線性相加。

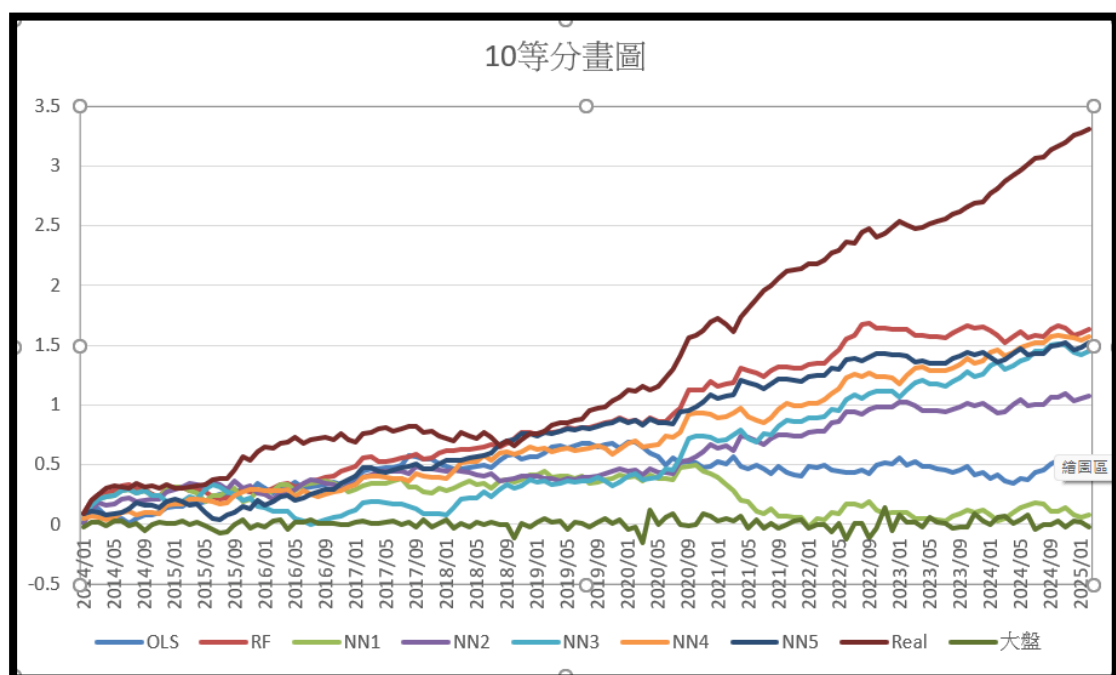
$$V_t = V_{t-1}(1 + r_t) \quad (11)$$

四、研究成果

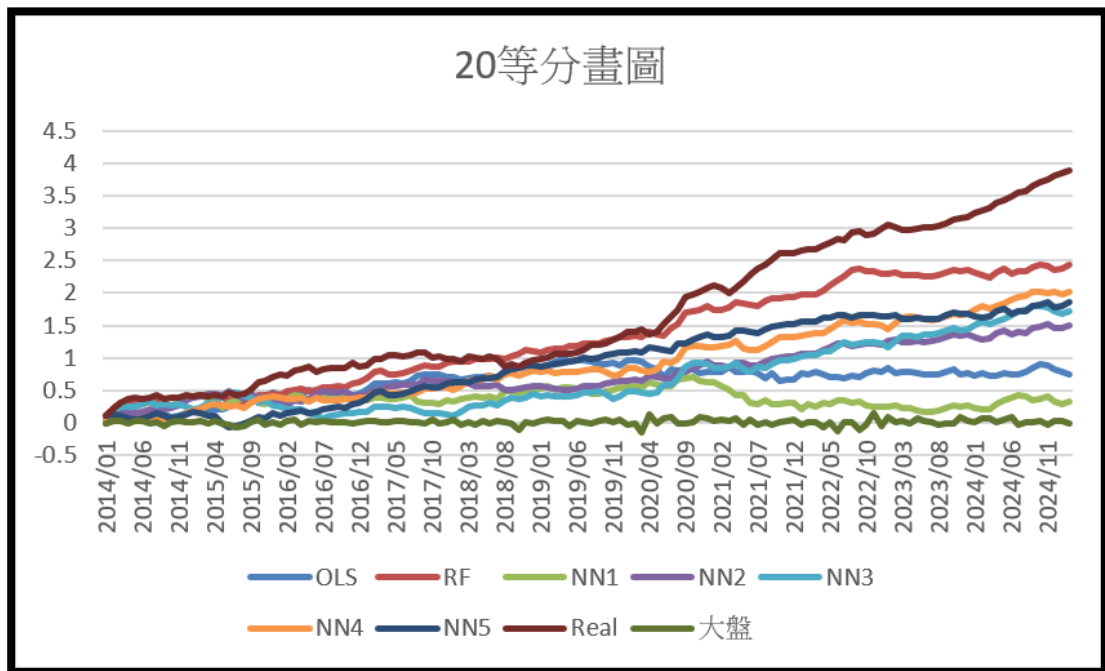
本研究利用三個不同機器學習模型將三因子深入分析，並利用累積報酬、勝率、夏普比率、樣本外的 R-square 以及再投資累積報酬等五種評估方法，找出最佳的投資組合，其研究成果將會以此五種評估方法作為切入的角度，來分析每個模型所預測出來的結果

4.1 累積報酬分析

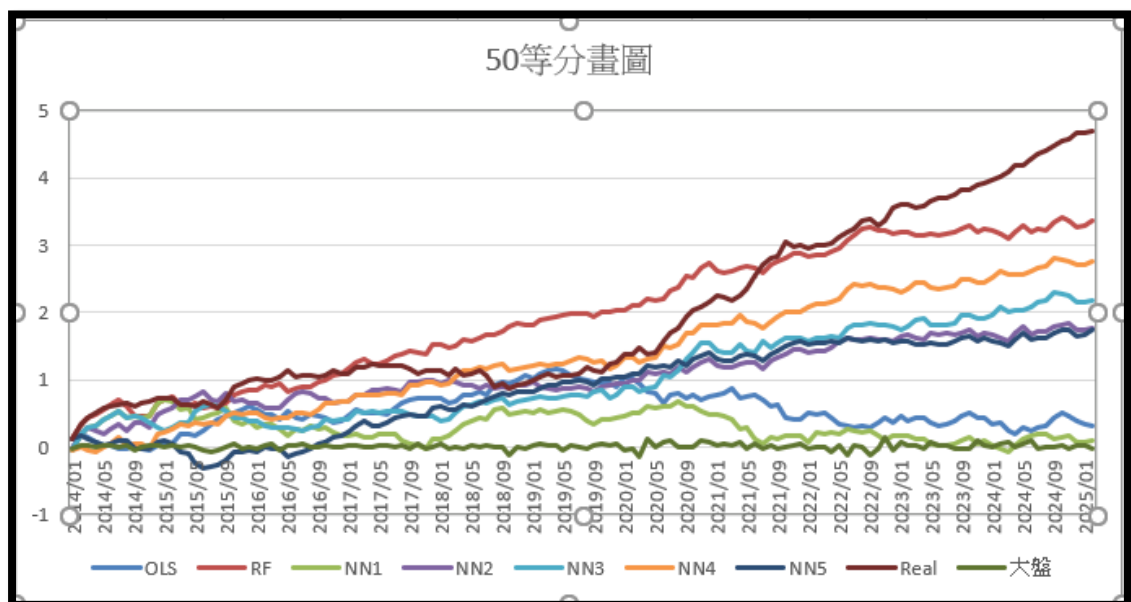
在累積報酬的分析中，我們將所有公司數分成 10、20、50、100、192 以及 964 這六個等分來進行分析，其每個等分的累積報酬折線圖，如圖 3(a)~(f)所示：



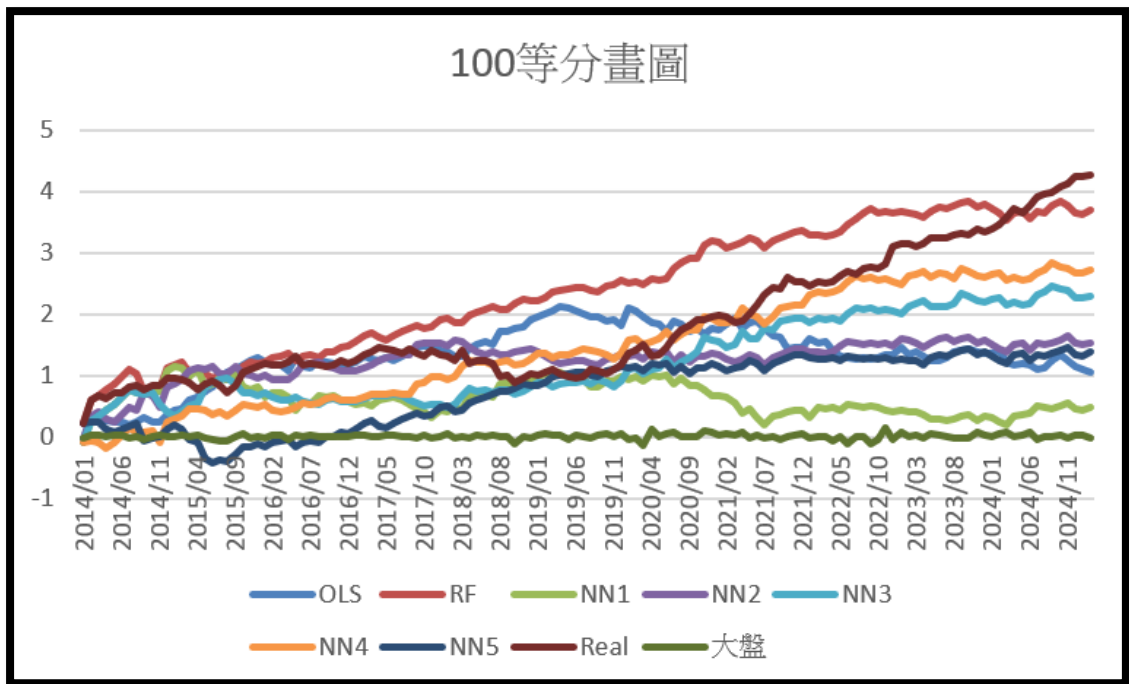
(a) 10 等分累積報酬



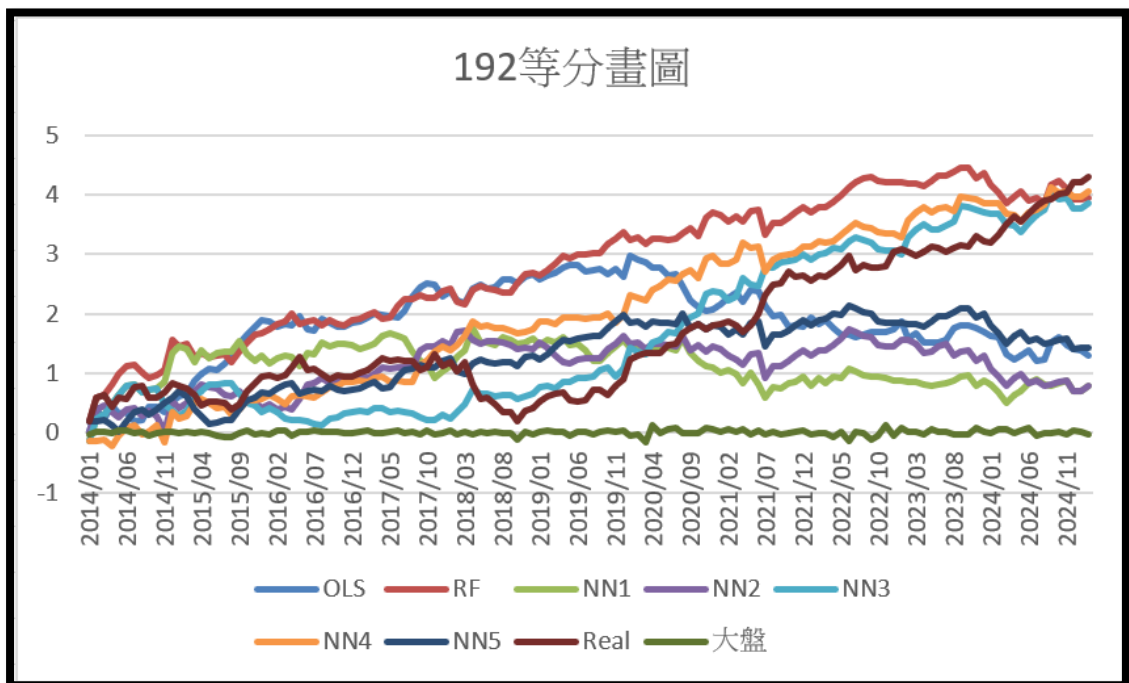
(b) 20 等分累積報酬



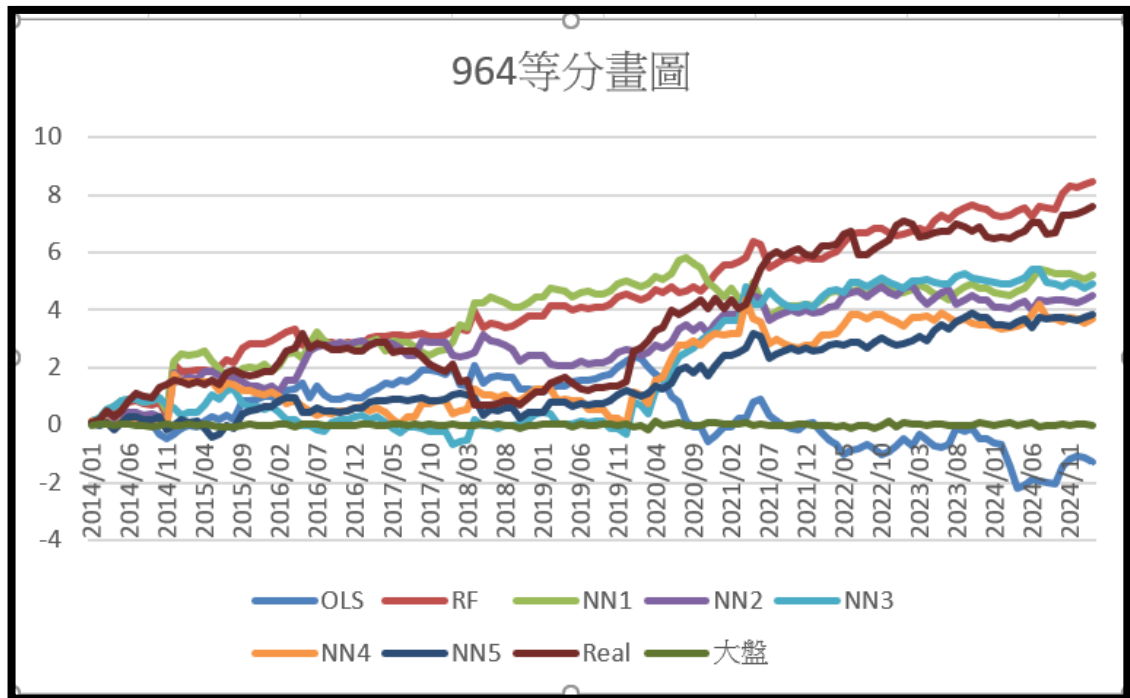
(c) 50 等分累積報酬



(d) 100 等分累積報酬



(e) 192 等分累積報酬



(f) 192 等分累積報酬

圖 3、不同模型與真實在每家數等分時的累積報酬

從 2014 年至 2025 年，不論將股票池切成 10、20、50、100、192 還是 964 等分，「真實報酬 (Real)」始終是表現最佳的一條曲線，而且對大盤（綠色）保持明顯優勢——在最粗的 10 等分下，Real 的最終累積值約為大盤的 3 倍；即使分到 964 等分、資料最稀疏時，Real 仍比大盤高出接近 8 個基準點，可見市場本身提供的多頭紅利仍遠大於被動持有大盤。相對於 Real，各模型的超額收益則呈現「分組越細，差距越縮」的趨勢：在 10 - 50 等分時，只有 NN4、RF 能稍微接近 Real；到 192、964 等分時，NN4、NN5 與 RF 的彎道超車現象變得明顯，尾盤區間幾乎能與 Real 並駕齊驅，顯示非線性模型在高維度、樣本更分散的情境下更能提取 Alpha，而 OLS 與 NN1 則因模型假設或深度不足，隨分組變細而逐漸落後甚至出現回撤。

綜合來看，「真實 > (RF ≈ NN4 ≈ NN5) > OLS > NN1 > 大盤」的排序大致貫穿各種切分尺度，只是隨著公司數增加，非線性模型的優勢愈發明顯，並縮短與 Real 之間的差距；而大盤始終位居末位，凸顯了主動選股（或高頻再平衡）在本資料期間確實可創造顯著超額報酬。這也提醒投資者：若要在更細分的股票池上維持領先，不僅需要複雜度更高的模型，還得持續監控其在極端市場波動時的穩健性與風險回撤。

4.2 勝率分析

各模型在不同家數等分下與大盤比較的勝率，真實操作（Real）始終維持在 0.60 左右的水準，不論把股票池切成 10 家還是 964 家，其每期打敗大盤的機率都顯著高於隨機擲銅板的 0.50。線性回歸（OLS）則長期徘徊在 0.47 - 0.51 之間，幾乎與隨機無異；最淺層的 NN1 更常落到 0.44 - 0.52，顯示在本資料期間，若只靠線性假設或過於簡單的網路結構，很難穩定捕捉 α 。相對地，隨機森林（RF）與較深層的 NN3 - NN5 隨「家數」增多而顯著提昇：RF 從 10 家時的 0.54 持續上升到 192 家時的 0.63，NN3 與 NN5 亦在大樣本、細分市場下突破 0.59，並多次逼近或短暫追平 Real，凸顯非線性與高容量模型在高維度環境的優勢。

家數/勝率	Real	OLS	RF	NN1
10	0.6119403	0.5	0.53731343	0.44776119
20	0.61940299	0.5	0.55223881	0.44029851
50	0.6119403	0.44776119	0.57462687	0.43283582
100	0.59701493	0.49253731	0.59701493	0.47014925
192	0.59701493	0.53731343	0.62686567	0.49253731
964	0.6194	0.50746269	0.6119403	0.51492537

家數/勝率	NN2	NN3	NN4	NN5
10	0.49253731	0.52985075	0.53731343	0.5
20	0.49253731	0.52238806	0.53731343	0.51492537
50	0.47761194	0.52985075	0.57462687	0.52985075
100	0.49253731	0.53731343	0.54477612	0.55970149
192	0.55223881	0.58955224	0.56716418	0.58955224
964	0.55223881	0.54477612	0.49253731	0.59701493

表 1、個個模型在不同家數等分下與大盤比較的勝率

4.3 夏普比率分析

其結果如表 2 所示，相對於大盤（夏普 ≈ 0.17 ），RF 是唯一在所有切分規模（10–964 家）始終保持高於基準、且隨家數變化仍能維持 0.25–0.44 的模型；Real 於 10–192 家都顯著優於大盤，但在 964 家時跌破 0 而首度落後；NN3、NN4 在 10–192 家維持 0.20–0.36、明顯勝出，到了 964 家則退到 0.15 以下而落後；NN2、NN5 僅在 10–50 家能以 0.22–0.36 打敗大盤，隨分組增多後便低於基準；至於 OLS 與 NN1 全程都停留在 0.01–0.13 區間，無論何種分組都遜於大盤。

家數/夏普比率	Real	OLS	RF	NN1
10	0.6172301	0.09253727	0.34492762	0.01607641
20	0.60181039	0.12386375	0.43701341	0.05300993
50	0.51663453	0.03554688	0.39986647	0.00965208
100	0.35796221	0.09024104	0.32602222	0.0379729
192	0.26385066	0.07931064	0.24534018	0.0460177
964	-0.03647708	-0.03647708	0.25619366	0.14314617

家數/夏普比率	NN2	NN3	NN4	NN5
10	0.23822698	0.27807205	0.34518308	0.34963331
20	0.2865962	0.27609484	0.3631058	0.3634849
50	0.21657741	0.25021501	0.34946673	0.23592019
100	0.1367588	0.20377894	0.25556238	0.13976586
192	0.0492081	0.2545804	0.25425153	0.10450501
964	0.13057612	0.14767559	0.09303153	0.14770928

大盤夏普比率	0.1738214
--------	-----------

表 2、各模型在不同等分以及大盤的夏普比率

4.4 樣本外 R-square 分析

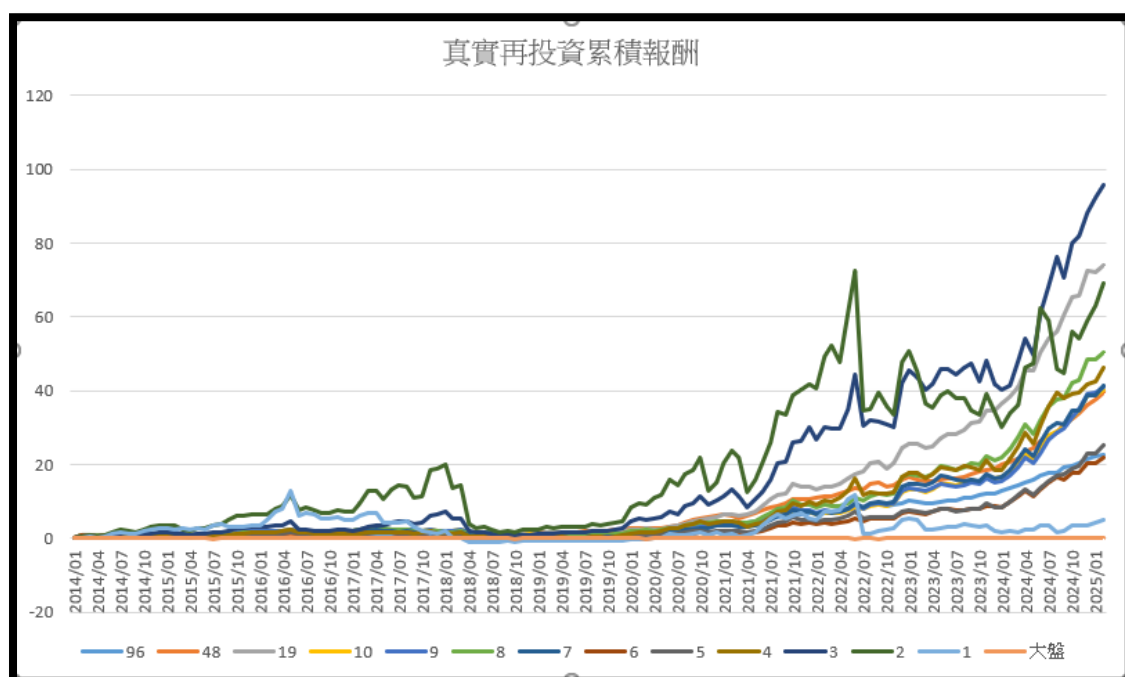
如表 3 所示，只有 RF (0.045) 與較深層的 NN4 (0.042)、NN3/NN5 (皆約 0.007) 勉強為正，代表它們在預測時僅比「用歷史平均值當預測」好一些；其他模型全為負，其中 OLS - 2.92 嚴重低於 0，顯示線性回歸高度過度擬合、預測表現甚至遠劣於隨機基準；NN1 - 0.24 與 NN2 - 0.005 亦無法有效泛化。

	OLS	RF	NN1
樣本外R-square	-2.92023396243308	0.0449739548688749	-0.236530294360169
NN2	NN3	NN4	NN5
-0.00475652	0.007276927	0.042088784	0.007276927

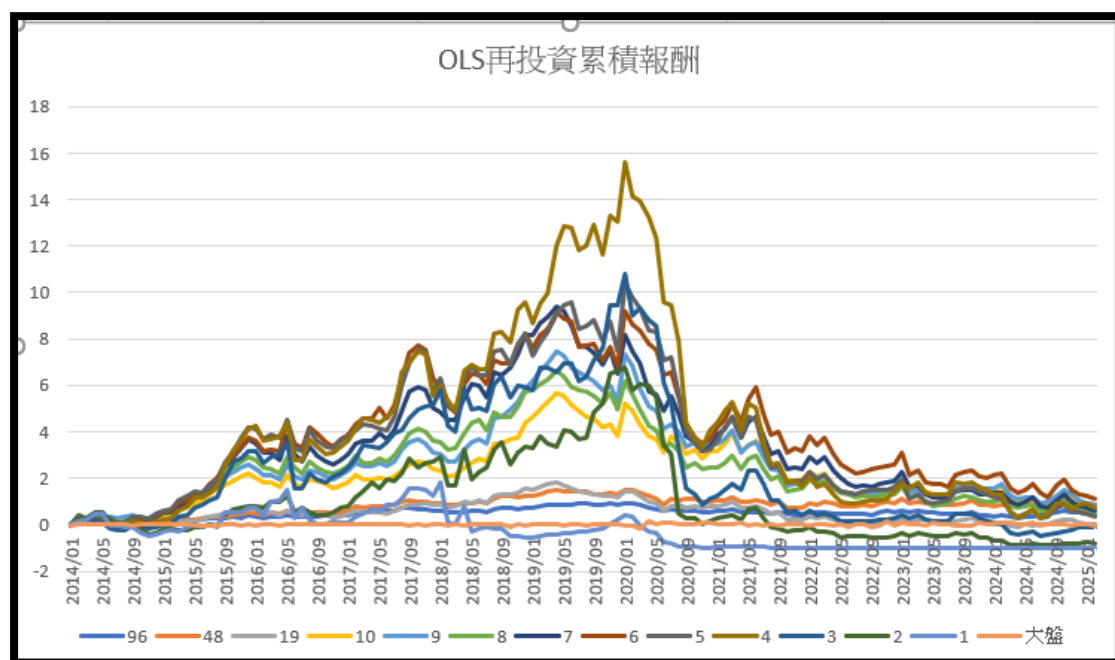
表 3、各模型樣本外 R-square 結果

4.5 再投資累積報酬率分析

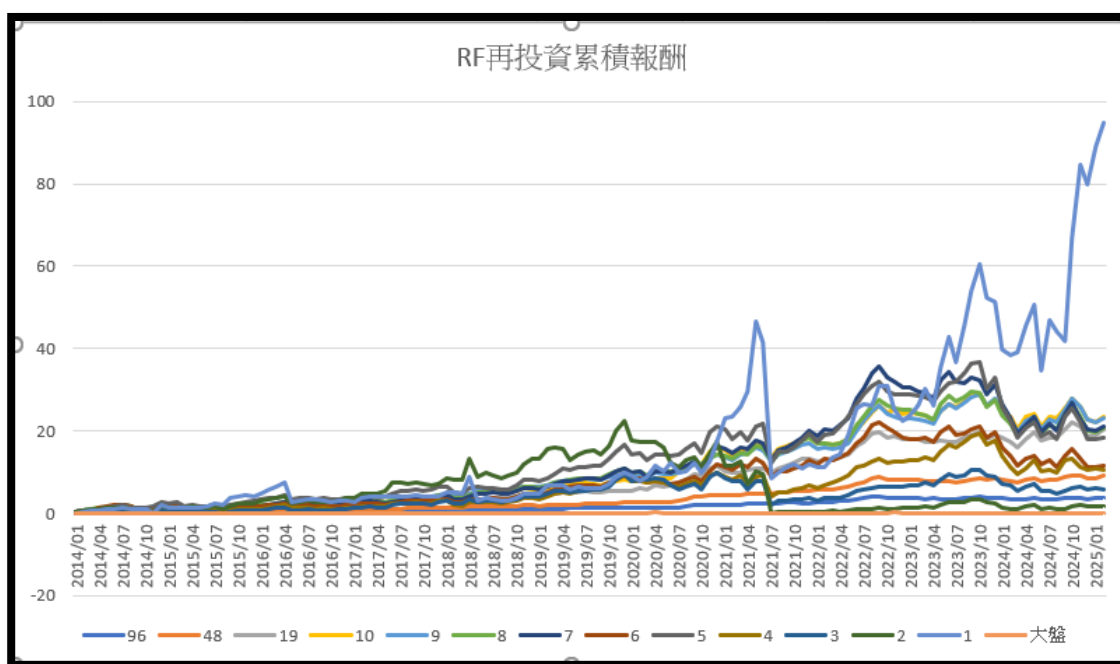
從圖 4(a)~(h)來看，「真實操作」與 RF 顯著領先所有模型：兩者在 2020 年後報酬曲線明顯轉陡，持股較多的組合（96、48 檔）衝幅最大，真實操作甚至突破百倍大關，RF 亦快速攀抵 90 倍上下。深層網路 NN3、NN4 雖不及前兩者，但自 2021 年起仍能穩健維持十多倍至二十多倍的漲幅，顯示隨模型容量增加而顯露的非線性優勢。相較之下，OLS 雖在 2018 - 2019 年短暫飆升，卻於 2020 年後急遽回吐，最終收益近乎歸零；淺層 NN1、NN2 亦僅在疫情前後出現尖峰，隨後便趨於平緩，整體僅累積數倍報酬。



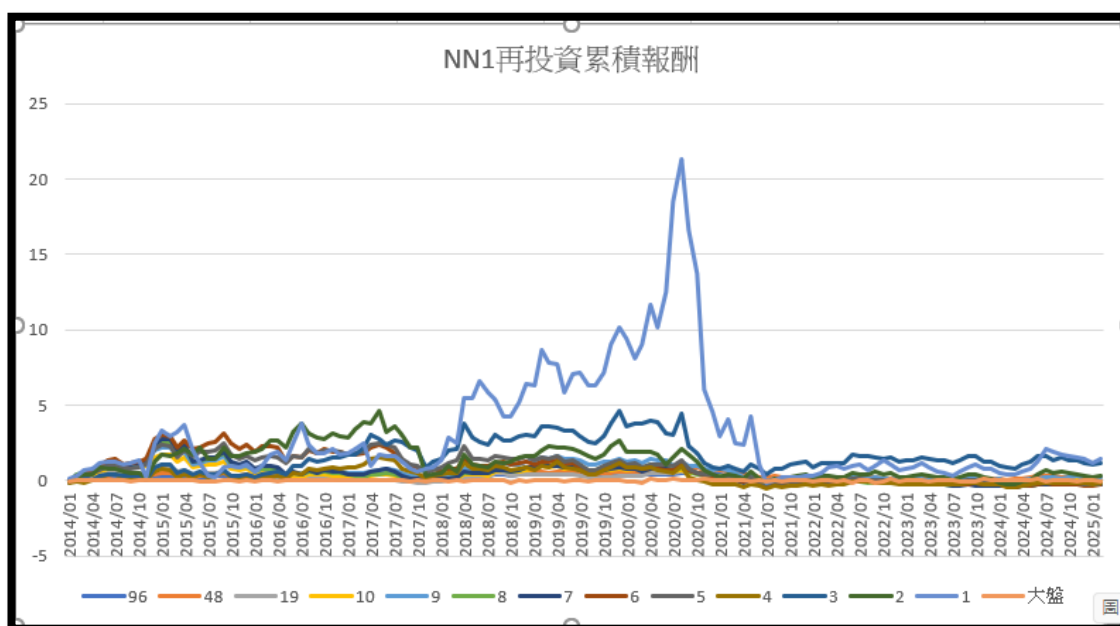
(a) 真實再投資累積報酬



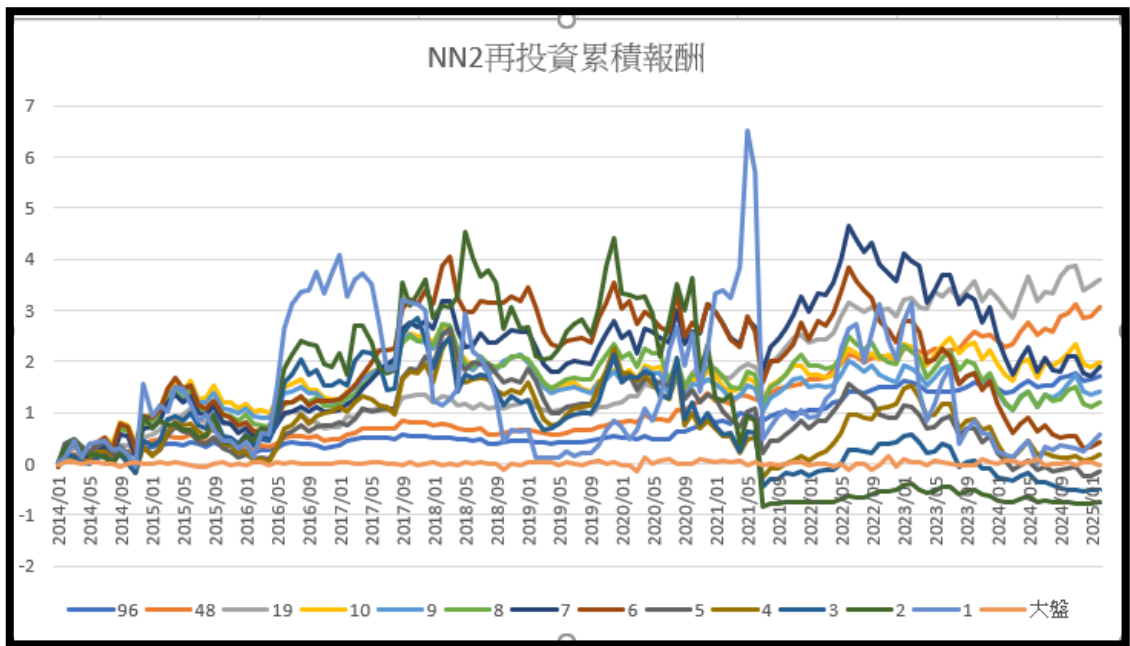
(b) OLS 再投資累積報酬



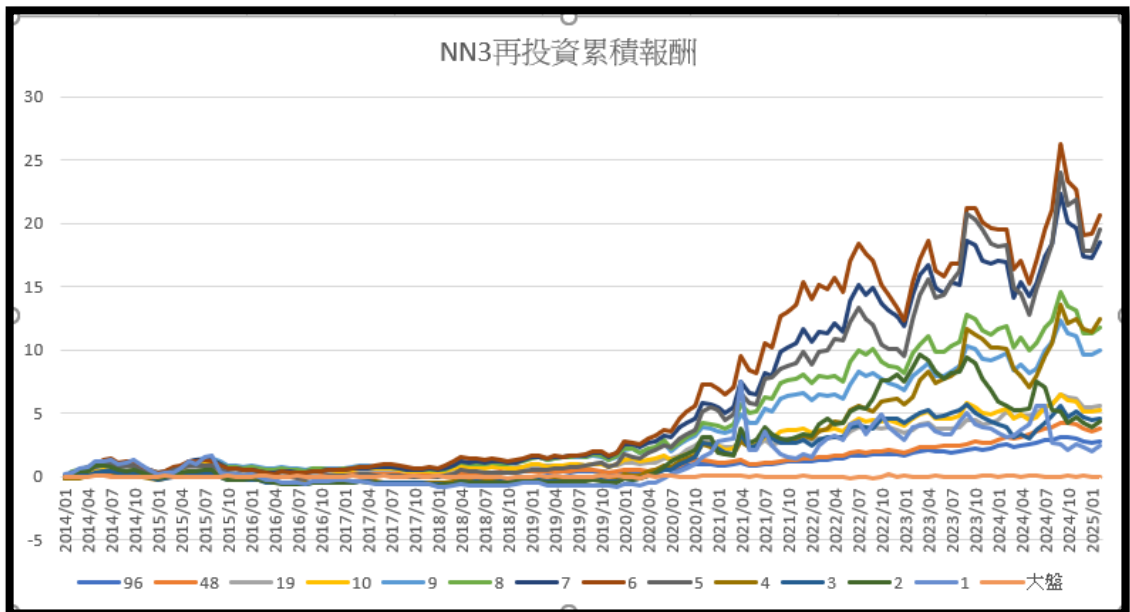
(c) RF 再投資累積報酬



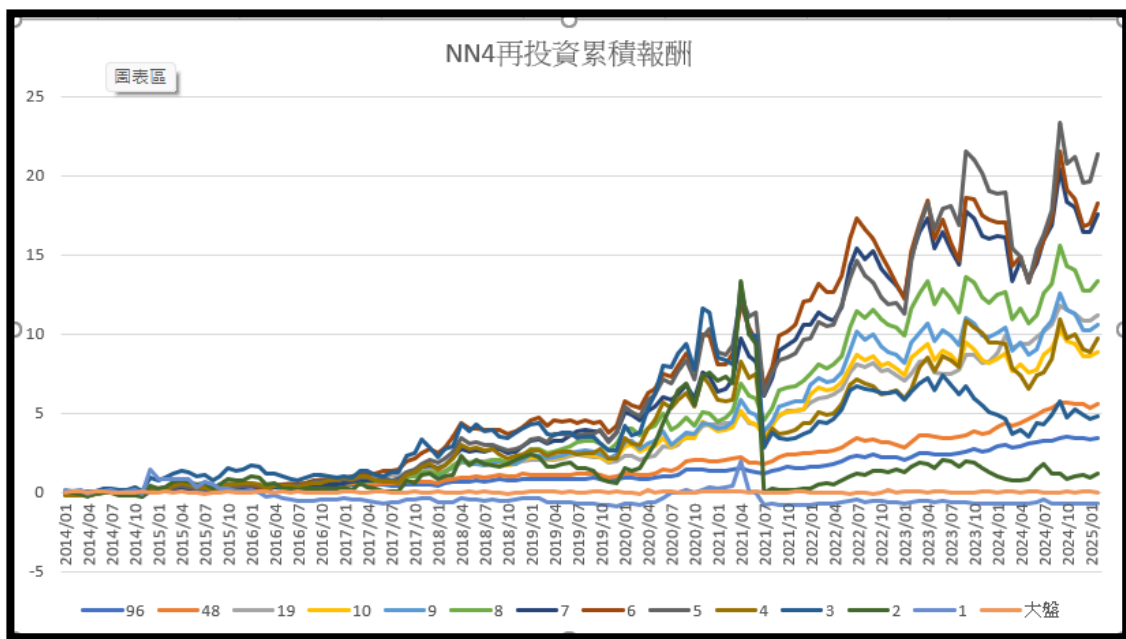
(d) NN1 再投資累積報酬



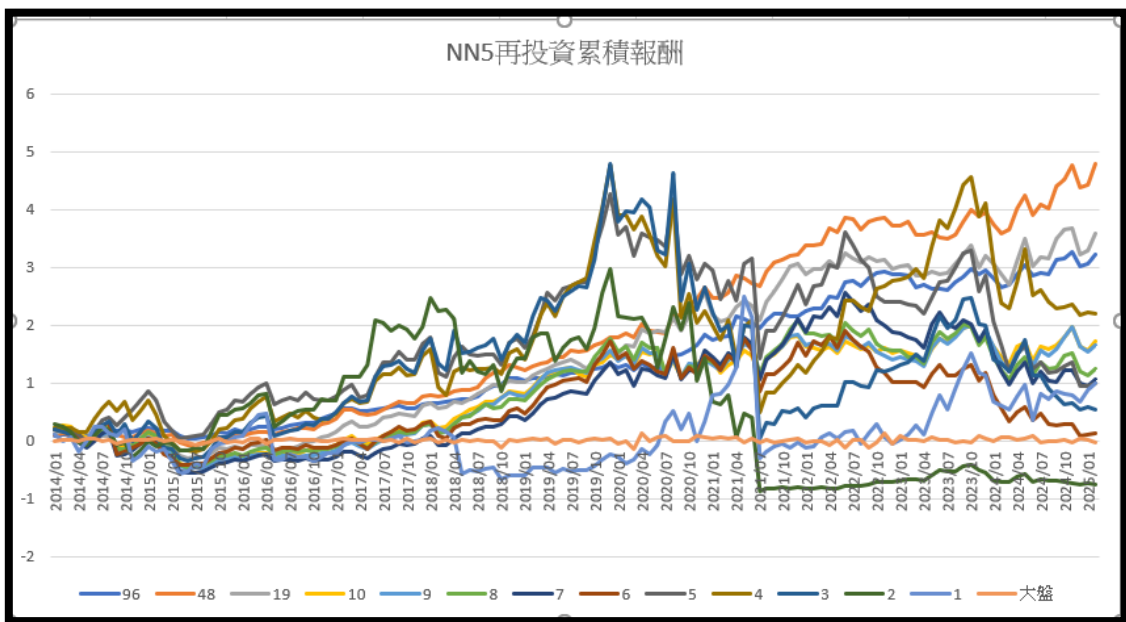
(e) NN2 再投資累積報酬



(f) NN3 再投資累積報酬



(g) NN4 再投資累積報酬



(h) NN5 再投資累積報酬

圖 4、各模型的再投資累積報酬

五、研究結論

本研究以臺灣上市櫃公司 2014–2025 年月度資料為樣本，結合「規模、帳面市值比、動能」三因子與機器學習模型，動態預測因子資訊係數並調整權重，實證檢視不同等份排序投組的績效。整體實驗結果清楚揭露三項關鍵洞見：第一，非線性模型優勢確立——隨機森林（RF）在所有持股數（10–964 檔）皆穩定跑贏大盤，不僅勝率長期維持 0.54–0.63，更是唯一在極端分散情境下仍具正夏普（ ≈ 0.26 ）的模型；深層神經網路（NN3–NN4、部分 NN5）亦能在 10–192 檔區間持續取得 0.20 以上的夏普與 0.55 左右的勝率，顯示較高模型容量對擷取複雜 α 因子至關重要。第二，線性與淺層模型侷限明顯——OLS 的樣本外 R-square 嚴重為負（-2.92），勝率僅貼近擲銅板，且其再投資曲線在 2020 年後迅速回吐；NN1、NN2 同樣缺乏泛化能力，無論勝率、夏普或累積報酬皆難超越大盤。最後，真實操作（Real）雖為絕對績效標竿，但對極度擴張的投組規模敏感：在 10–192 檔時保持最高勝率（ ≈ 0.60 ）與夏普（0.26–0.62），卻在 964 檔時風險調整後報酬轉負，提醒實務運用中仍需關注流動性與執行成本。

與傳統線性因子模型相比，透過機器學習預測 IC 並動態權重配置，確實可在長期投資中顯著提高勝率、強化報酬穩定性，並大幅超越臺灣加權股價指數。其中，RF 兼具可解釋性與穩健度，是最具全周期優勢的模型；深層 NN 可視為在中等持股數下的進階選擇；而 OLS 與淺層 NN 僅適合作為基線比較。綜合而言，本研究驗證了「因子投資 $\times$ AI 預測」的可行性與必要性：當市場結構愈趨複雜、訊號愈呈非線性時，引入資料驅動、能自動萃取高階特徵的模型，方能持續創造 α 並降低尾端風險，為資產管理領域提供具體且可操作的實務指引。

六、心得感想

以前的我對金融真的沒什麼興趣，也覺得自己應該不會碰這一塊。當時的想法是：只要專注在自己想做的事情上，就算不投資，也可以把生活過得不錯。但上了這門課之後，我的想法有了滿大的轉變。

當我開始研究金融怎麼跟機器學習結合的時候，才發現其實金融沒我想得那麼複雜。它有點像在解數學題，只要肯花時間練習、多思考，慢慢就能掌握一些判斷的邏輯，也能做出更好的決策，甚至有機會獲得實質的回報。

這門課不只讓我對金融有了新的認識，也讓我開始對這個領域感興趣。我希望之後可以繼續學習更多相關的知識，看看自己在金融和機器學習結合的這條路

上，還能走多遠。

七、參考文獻

- [1] W. F. Sharpe, *CAPITAL ASSET PRICES: A THEORY OF MARKET EQUILIBRIUM UNDER CONDITIONS OF RISK*. 1964.
- [2] E. F. Fama and K. R. French, *The Cross-Section of Expected Stock Returns*. *THE JOURNAL OF FINANCE*, 1992. [Online]. Available: <https://onlinelibrary.wiley.com/doi/epdf/10.1111/j.1540-6261.1992.tb04398.x>
- [3] M. M. CARHART, *On Persistence in Mutual Fund Performance*. *THE JOURNAL OF FINANCE*, 1997. [Online]. Available: <https://onlinelibrary.wiley.com/doi/epdf/10.1111/j.1540-6261.1997.tb03808.x>
- [4] S. Gu, B. Kelly, and D. Xiu, *Empirical Asset Pricing via Machine Learning*. *Review of Financial Studies*, 2020. [Online]. Available: <https://doi.org/10.1093/rfs/hhz077>
- [5] B. Kelly and D. Xiu, *Machine Learning for Asset Pricing*. *Annual Review of Financial Economics*, 2022. [Online]. Available: <https://doi.org/10.1146/annurev-financial-110821-120750>
- [6] S. Titman, *Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency*. *Journal of Finance*, 1993.
- [7] D. G. Bui, D. R. Kong, and C. Y. Lin, *Momentum in machine learning: Evidence from the Taiwan stock market*.
- [8] G. Feng, H. He, and D. Xiu, *Deep learning in asset pricing*. *Review of Financial Studies*, 2020.
- [9] J. Sirignano and R. Cont, *Universal features of price formation in financial markets*. *Perspectives from deep learning*. *Quantitative Finance*, 2019.

八、附錄(程式碼說明)

Github 連結:

https://github.com/SHUWEI-HO/SCU_HW.git (Finetech)

1. main.py

主要啟動程式，其包含了利用三種模型去計算 IC 因子、投資組合、勝率、夏普比率、設定儲存格式等流程

```

main.py > main
1  import os
2  import argparse
3  from tqdm import tqdm, trange
4  from utils.ic_calculate import IC
5  from utils.tools import *
6
7
8  '''
9  執行程式碼前須要注意是否有真實ic跟預測ic的表格樣式
10 '''
11
12 os.environ["CUDA_VISIBLE_DEVICE"] = "0"
13
14
15
16 def main() -> None:
17     args = get_parser()
18     input_dir = args.directory
19     output = os.path.join(input_dir, args.save_file)
20     model_name = args.model
21     date_str = args.date
22     n_jobs = args.n_jobs if args.n_jobs > 0 else os.cpu_count() * 10
23
24     data_dict = load_worksheet(input_dir, output) # 讀入所有 Excel
25     print(f"呼叫 {model_name} model (n_jobs = {n_jobs}) ")
26     """
27     times 是 RandomForest中的random_state參數
28     為了驗證最佳報酬所給定的範圍(如果只想做一次, 則times = 1

```

```

為了驗證最佳報酬所給定的範圍(如果只想做一次，則times = 1
然後RF model設定則為random_state = 999)
"""

# 讀取大盤資料
ic_ws = data_dict["IC"]["預測IC"]
mrow, mcol = find_element(ic_ws, "大盤")
market = [
    list(ic_ws.values)[mrow - 1][mcol + 1 + col_idx]
    for col_idx in trange(134, desc="讀取大盤")
]

if not args.arr:

    start = int(args.range[0]) if model_name == "RF" else 1
    times = int(args.range[1]) if model_name == "RF" else 1
    last_data = None

    for random_state in range(start, times + 1):
        print(f"Times:{random_state}/{times}")

        random_state = 1042 if start == times else random_state
        # 計算所有因子的IC
        total_results = IC(input_dir, output, model_name, n_jobs).process(random_state)

        # 選擇基準因子
        data_storage, id_storage, select_values = select_factor(total_results)

```

```

date_str = args.date
# 計算勝率與累積報酬
portfolio_value, total_count, win_rates = win_rate(portfolio_data, market)
cumulative_rewards = cumulative_reward(portfolio_data)
select_reward = max(
    [cumulative_rewards[i][-1] for i in range(len(cumulative_rewards))]
)

candidate = dict(
    IC=data_storage,
    ID=id_storage,
    select_values=select_values,
    portfolio_data=portfolio_data,
    portfolio_value=portfolio_value,
    total_count=total_count,
    win_rate=win_rates,
    cumulative_rewards=cumulative_rewards,
    select_reward=select_reward,
    best_reward=random_state,
)

if times == 1:
    last_data = candidate

current_reward = last_data["select_reward"] if last_data is not None else None
current_state = last_data["best_reward"] if last_data is not None else None
if model_name == "RF":

```

```

    if model_name == "RF":
        print(f"目前設定參數為{random_state}，其累積報酬率(select_reward)，目前最佳參數(current_
# 如果新的預測結果比目前的存取的最佳紀錄就更新，方便之後可以直接用最佳數據存取
if last_data is None or select_reward > last_data["select_reward"]:
    if model_name == "RF":
        print(f"在參數{random_state}有更好的累積報酬率")
        last_data = candidate

best_state = last_data["best_reward"]
best_reward = last_data["select_reward"]

max_records, min_records, max_company, min_company = find_company(input_dir, id_storage)

last_data['max_records'] = max_records
last_data['max_company'] = max_company
last_data['min_records'] = min_records
last_data['min_company'] = min_company

if model_name == "RF":
    print(f"最後選定參數為:{best_state}，其最佳累積報酬率:{best_reward}")

# 計算夏普比率
sharpe_data, sharpe_market = sharpe_ratio(portfolio_data, market)
last_data["sharpe_ratio"] = sharpe_data
last_data["sharpe_ratio_market"] = sharpe_market

```

```

def main() -> None:
    last_data['max_records'] = max_records
    last_data['max_company'] = max_company
    last_data['min_records'] = min_records
    last_data['min_company'] = min_company

    if model_name == "RF":
        print(f"最後選定參數為:{best_state}，其最佳累積報酬率:{best_reward}")

    # 計算夏普比率
    sharpe_data, sharpe_market = sharpe_ratio(portfolio_data, market)
    last_data["sharpe_ratio"] = sharpe_data
    last_data["sharpe_ratio_market"] = sharpe_market

    # 計算R-square
    real_ic = list(data_dict["IC"]["真實IC"].values)
    real_bm = real_ic[2][2:]
    real_sz = real_ic[3][2:]
    real_mm = real_ic[4][2:]

    real_ic = [[bm, sz, mm] for bm, sz, mm in zip(real_bm, real_sz, real_mm)]

    pr, rr, RS = r_square(real_ic, data_storage)

    last_data["PR_RS"] = pr
    last_data["RR_RS"] = rr

```

```

else:
    ...
    由於Real.xlsx並沒有計算累積報酬、勝率等訊息
    因此在這邊會先計算完，如果存在還是會再重新算一次，不會影響到後續的數值
    ...

    real_data = openpyxl.load_workbook(os.path.join(input_dir, 'Real.xlsx'), data_only=True)

    real_portfolio_data = [[list(real_data["預測IC"].values)[11+c_idx][2+r_idx]
                             for c_idx in range(13)] for r_idx in range(134) ]

    real_portfolio_value, real_total_count, real_win_rates = win_rate(real_portfolio_data, market)
    real_cumulative_reward = cumulative_reward(real_portfolio_data)
    real_sharpe_data, sharpe_market = sharpe_ratio(real_portfolio_data, market)

    data = {
        "win_rate":real_win_rates,
        "portfolio_value":real_portfolio_value,
        "total_count": real_total_count,
        "cumulative_rewards":real_cumulative_reward,
        "sharpe_ratio":real_sharpe_data,

```

```

    real_portfolio_value, real_total_count, real_win_rates = win_rate(real_portfolio_data, market)
    real_cumulative_reward = cumulative_reward(real_portfolio_data)
    real_sharpe_data, sharpe_market = sharpe_ratio(real_portfolio_data, market)

    data = {
        "win_rate":real_win_rates,
        "portfolio_value":real_portfolio_value,
        "total_count": real_total_count,
        "cumulative_rewards":real_cumulative_reward,
        "sharpe_ratio":real_sharpe_data,
        "sharpe_ratio_market": sharpe_market
    }

    real_data_store(os.path.join(input_dir, 'Real.xlsx'), real_data, data)

    ...
    再投資累積報酬
    將(IC值+1)*(下一期IC值+1)...累乘後-1即為最後的再投資累積報酬率
    ...

    ELEMENT_ID = ["Real", "OLS", "RF", "NN1", "NN2", "NN3", "NN4", "NN5"]
    data_dict = load_worksheet(inputs=input_dir, element_id=ELEMENT_ID)

    for name, wb in tqdm(data_dict.items(), total=len(data_dict), desc='計算再投資累積報酬...'):

```

```

def main() -> None:
    將(IC值+1)*(下一期IC值+1)...累乘後-1即為最後的再投資累積報酬率

    ...

    ELEMENT_ID = ["Real", "OLS", "RF", "NN1", "NN2", "NN3", "NN4", "NN5"]
    data_dict = load_worksheet(inputs=input_dir, element_id=ELEMENT_ID)

    for name, wb in tqdm(data_dict.items(), total=len(data_dict), desc='計算再投資累積報酬...'):
        portfolio_data = [[list(wb["預測IC"].values)[11+c_idx][2+r_idx]
                             for c_idx in range(13)] for r_idx in range(134) ]

        arr_cumulative(wb, name, portfolio_data, market, input_dir)

    print("計算完成!!!")

def get_parser() -> argparse.Namespace:
    p = argparse.ArgumentParser("金融投資策略")
    p.add_argument("--directory", default="../data/", help="資料資料夾")
    p.add_argument("--save-file", default="NN1.xlsx", help="輸出檔名")
    p.add_argument("--date", default="2013/12", help="起始日期")
    p.add_argument("--range", nargs=2, default=[1, 1], help="設置要跑的參數範圍")
    p.add_argument(
        "--model", choices=["LR", "RF", "NN"], default="LR", help="選擇模型種類"
    )
    p.add_argument('--arr', action='store_true', help='計算所有的再投資累積報酬率')
    p.add_argument(

```

2. ic_calculate.py

這裡主要是計算 IC 的過程，將 bm、size；mom 三因子的結果以字典行是包起來回傳給 main.py

```

import os
import warnings
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import Dict, List
import pandas as pd
import random
from tqdm import tqdm
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from utils.module import neural_network
from utils.tools import *

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
warnings.filterwarnings("ignore")

import tensorflow as tf

tf.get_logger().setLevel("FATAL")
try:
    tf.autograph.set_verbosity(0)
except AttributeError:
    pass

SEED = 1042

os.environ["OMP_NUM_THREADS"] = str(os.cpu_count())
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
os.environ["PYTHONHASHSEED"] = str(SEED)

MODEL_MAP = {
    "LR": LinearRegression,
    "RF": RandomForestRegressor,

```

SHUWEL-HQ (2 週前)


```

MODEL_MAP = {
    "LR": LinearRegression,
    "RF": RandomForestRegressor,
    "NN": "NN",
}

TARGETS = ["bm", "size", "mom"]

class IC:
    def __init__(
        self,
        directory: str,
        outputs: str,
        model_name: str,
        n_jobs: int = os.cpu_count(),
    ) -> None:
        self.dir = directory
        self.output = outputs
        self.n_jobs = n_jobs
        self.model_cls = MODEL_MAP[model_name]
        # For NeuralNetwork
        if self.model_cls == "NN":
            tf.keras.utils.disable_interactive_logging()
            self.model, self.early_stop = self._make_model()

    def _make_model(self, random_state: int = 0):
        if self.model_cls is RandomForestRegressor:
            return self.model_cls(
                max_depth=3,
                random_state=random_state,
                n_estimators=100,
                n_jobs=self.n_jobs,
            ), None

```

SHIWEI-HO (23)

```

def _make_model(self, random_state: int = 0):
    model, early_stop = neural_network(input_dim=964)
    return model, early_stop

    return self.model_cls(), None

def process(self, random_state: int = 999) -> Dict[str, List[float]]:
    total_results: Dict[str, List[float]] = {}

    def _fit_predict(i: int, idx: int, neural_network: bool = False) -> float:

        model, _ = self._make_model(random_state)
        X = df_x.iloc[idx].values
        y = df_y.iloc[idx].values
        tx = df_x.iloc[idx : idx + 1].values
        model.fit(X, y)
        return i, model.predict(tx)[0]

    if self.model_cls == "NN":
        for file_name in tqdm(TARGETS, total=len(TARGETS), desc="計算因子"):
            input_path = os.path.join(self.dir, f"{file_name}.xlsx")
            df_x = pd.read_excel(input_path, sheet_name=f"{file_name}補値").T
            df_y = pd.read_excel(input_path, sheet_name=f"{file_name}IC").T
            df_x.columns, df_y.columns = df_x.iloc[0], df_y.iloc[1]
            df_x, df_y = df_x.iloc[2:], df_y.iloc[2:]

            total_run = len(df_x) - 1
            start_time = 166 if file_name == "mom" else 178
            n0 = 48
            results = []
            for idx in range(start_time, total_run):
                X = df_x.iloc[:idx].values.astype(np.float64)
                y = df_y.iloc[1 : idx + 1].values.ravel().astype(np.float64)
                tx = df_x.iloc[idx : idx + 1].values.astype(np.float64)

```

```

if self.model_cls == "NN":
    for file_name in tqdm(TARGETS, total=len(TARGETS), desc="計算因子"):
        input_path = os.path.join(self.dir, f"{file_name}.xlsx")
        df_x = pd.read_excel(input_path, sheet_name=f"{file_name}補値").T
        df_y = pd.read_excel(input_path, sheet_name=f"{file_name}IC").T
        df_x.columns, df_y.columns = df_x.iloc[0], df_y.iloc[1]
        df_x, df_y = df_x.iloc[2:], df_y.iloc[2:]

        total_run = len(df_x) - 1
        start_time = 166 if file_name == "mom" else 178
        n0 = 48
        results = []
        for idx in range(start_time, total_run):
            X = df_x.iloc[:idx].values.astype(np.float64)
            y = df_y.iloc[1 : idx + 1].values.ravel().astype(np.float64)
            tx = df_x.iloc[idx : idx + 1].values.astype(np.float64)

            X_tr, X_vl = X[:idx-n0], X[idx-n0:idx]
            y_tr, y_vl = y[:idx-n0], y[idx-n0:idx]

            self.model.fit(
                X_tr,
                y_tr,
                validation_data=(X_vl, y_vl),
                epochs=100,
                batch_size=32,
                verbose=0,
                callbacks=[self.early_stop])

            pred = self.model.predict(tx)
            results.append(pred.tolist()[0][0])

```

```

else:
    for file_name in TARGETS:
        input_path = os.path.join(self.dir, f"{file_name}.xlsx")
        df_x = pd.read_excel(input_path, sheet_name=f"{file_name}補值").T
        df_y = pd.read_excel(input_path, sheet_name=f"{file_name}IC").T
        df_x.columns, df_y.columns = df_x.iloc[0], df_y.iloc[1]
        df_x, df_y = df_x.iloc[2:], df_y.iloc[2:]

        total_run = len(df_x) - 1
        start_time = 166 if file_name == "mom" else 178
        idx_range = list(range(start_time, total_run))
        results = [None] * len(idx_range)
        with ThreadPoolExecutor(max_workers=self.n_jobs) as executor:
            futures = [
                executor.submit(_fit_predict, i, idx)
                for i, idx in enumerate(idx_range)
            ]
            for f in tqdm(
                as_completed(futures),
                total=len(futures),
                desc=f"計算{file_name} IC因子 ",
            ):
                i, pred = f.result()
                results[i] = pred

        total_results[file_name] = results

    return total_results

```

3. utils/tools.py

在 tools.py 中，包含許多計算及儲存格式的函式

- fine_lemnet 在工作表找出值的位置

```

ELEMENT_ID: Dict[int, str] = {1: "bm", 2: "size", 3: "mom"}
SPLIT_NUM: List[int] = [96, 48, 19, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
MODEL_NAME: Dict[str, str] = {"LR": "OLS", "RF": "RF", "NN": "NN"}

def find_element(
    ws: Worksheet,
    target: str
) -> Tuple[Optional[int], Optional[int]]:

    row_pos = None
    col_pos = None
    for r_idx, row in enumerate(ws.iter_rows(values_only=True), 1):
        row_values = [str(v) for v in row]
        if target in row_values:
            row_pos = r_idx
            col_pos = row_values.index(target) + 1
            break
    return row_pos, col_pos

```

- select_factor 從每一其中篩選出絕對值最大的因子

```

def select_factor(data: Dict[str, List[float]]) -> Tuple[List[List[float]], List[int], List[float]]:

    lengths = {len(data[k]) for k in ELEMENT_ID.values()}
    if len(lengths) != 1:
        raise ValueError("計算錯誤，請檢查預測是否有誤")

    data_storage = [
        [bm, size, mom] for bm, size, mom in zip(data["bm"], data["size"], data["mom"])
    ]

    id_storage = []
    select_value = []
    for rec in tqdm(data_storage, desc="篩選因子"):
        idx: int = max(range(3), key=lambda i: abs(rec[i])) + 1
        id_storage.append(idx)
        select_value.append(float(rec[idx - 1]))

    return data_storage, id_storage, select_value

```

- load_workbook 開啟所有檔案，以字典回傳

```
def load_worksheet(inputs: str, output: str = None, element_id: List = None) -> Dict[str, Workbook]:
    if element_id is None:
        if isinstance(ELEMENT_ID, dict):
            data_dict: Dict[str, Workbook] = {
                v: openpyxl.load_workbook(os.path.join(inputs, f"{v}.xlsx"), data_only=True)
                for v in ELEMENT_ID.values()
            }

            data_dict["IC"] = openpyxl.load_workbook(
                '../data/IC.xlsx', data_only=True
            )
        else:
            if isinstance(element_id, list):
                data_dict: Dict[str, Workbook] = {
                    v: openpyxl.load_workbook(os.path.join(inputs, f"{v}.xlsx"), data_only=True)
                    for v in element_id
                }
            else:
                data_dict: Dict[str, Workbook] = {}

    return data_dict
```

- portfolio_calculate 計算投資組合

```
def portfolio_calculate(
    data_dict: Dict[str, Workbook],
    value: float,
    element_id: int,
    data_storage: List[List[float]],
    date_str: str = "2013/12",
) -> Tuple[str, List[List[float]]]:
    element_name = ELEMENT_ID[element_id]
    wb = data_dict[element_name]
    ws = wb[f"{element_name}補值"]
    nr = wb["下個月月報酬補值"]

    _, col = find_element(ws, date_str)
    if col is None:
        raise ValueError(f"在工作表中找不到日期: {date_str}")
    col -= 1

    ws_values = list(ws.values)
    nr_values = list(nr.values)

    def _process(col_idx: int, v: float) -> List[float]:
        pairs = sorted(
            (
                (ws_values[r][col_idx], nr_values[r][col_idx])
                for r in range(1, len(ws_values) - 1)
            ),
            key=lambda p: p[0],
        )
        sorted_nr = [p[1] for p in pairs]
        high_mean = [statistics.mean(sorted_nr[-sp:]) for sp in SPLIT_NUM]
        low_mean = [statistics.mean(sorted_nr[:sp]) for sp in SPLIT_NUM]
        return [(h - 1) if v >= 0 else (1 - h) for h, l in zip(high_mean, low_mean)]
```

```

def _process(col_idx: int, v: float) -> List[float]:
    pairs = sorted(
        (
            (ws_values[r][col_idx], nr_values[r][col_idx])
            for r in range(1, len(ws_values) - 1)
        ),
        key=lambda p: p[0],
    )
    sorted_nr = [p[1] for p in pairs]
    high_mean = [statistics.mean(sorted_nr[-sp:]) for sp in SPLIT_NUM]
    low_mean = [statistics.mean(sorted_nr[:sp]) for sp in SPLIT_NUM]
    return [(h - 1) if v >= 0 else (1 - h) for h, l in zip(high_mean, low_mean)]

with ThreadPoolExecutor(max_workers=os.cpu_count() * 100) as ex:
    data_storage.extend(ex.map(_process, [col], [value]))

y, m = map(int, date_str.split("/"))
m = m + 1 if m < 12 else 1
y = y if m != 1 else y + 1
next_date: str = f"{y}/{m:02d}"
return next_date, data_storage

```

- win_rate 計算勝率

```

def win_rate(
    portfolio_data: List[List[float]],
    market: List[float],
) -> Tuple[List[List[int]], List[int], List[float]]:

    if len(portfolio_data) != len(market):
        raise ValueError("數據不對稱，請檢查數據是否筆數一致")

    pf_vs_mk = [
        [int(p > mk) for p in pf]
        for pf, mk in tqdm(
            zip(portfolio_data, market),
            total=len(market),
            desc="計算勝率",
        )
    ]

    totals = [sum(col) for col in zip(*pf_vs_mk)]
    rates = [t / len(pf_vs_mk) for t in totals]
    return pf_vs_mk, totals, rates

```

- cumulative_reward 計算累積報酬

```
def cumulative_reward(data: List[List[float]]) -> List[List[float]]:
    col_iter = zip(*data)
    return [
        list(itertools.accumulate(col))
        for col in tqdm(
            col_iter, total=len(data[0]), desc="計算累積報酬"
        )
    ]
```

- sharpe_ratio 計算夏普比率

```
def sharpe_ratio(data: Sequence[Sequence[float]], market: List[float]) -> List[float]:
    sharpe = []

    for col in zip(*data):
        means = mean(col)
        standard = stdev(col)
        sharpe.append([means, standard, means / standard if standard else float("nan")])

    market_means = mean(market)
    market_standard = stdev(market)
    market_sharpe_ratio = (
        market_means / market_standard if market_standard else float("nan")
    )

    sharpe_market = []
    sharpe_market.append([market_means, market_standard, market_sharpe_ratio])

    return sharpe, sharpe_market
```

- ensure_sheet 確認工作表，若不存在則建立

```
def ensure_sheet(wb: Workbook, sheet_name: str) -> Worksheet:
    if sheet_name not in wb.sheetnames:
        return wb.create_sheet(sheet_name)
    return wb[sheet_name]
```


- r_square 計算樣本外 R-square

```
def r_square(real: List[List[float]], pred: List[List[float]]) -> List[List[float]]:

    real = np.array(real)
    pred = np.array(pred)

    rs_rp = np.square(pred - real)
    rs_rr = np.square(real)

    RS = 1 - float(np.sum(rs_rp)/ np.sum(rs_rr))

    return rs_rp.tolist() , rs_rr.tolist(), RS
```

- classifier_store 累積報成等分統整(10、20、50、...等分)

```
def classifier_store(
    predict_cumulative_reward: List[List[List]],
    real_cumulative_reward: List[List],
    market: List,
    element_id: List,
    output: Path,
    date: List
)-> None:

    # 創建新的工作表
    wb = Workbook()
    default_sheet = wb.active
    wb.remove(default_sheet)

    select_dict = {0:10, 1:20, 2:50, 3:100, 8:192, 12:964}

    # 不需要將13次的分類，只需要取10、20、50、100、192跟964的等分的錄選
    for idx, (pcrs, rcr) in enumerate(zip(predict_cumulative_reward, real_cumulative_reward)):
        (parameter) real_cumulative_reward: List[List] ),
        real_cumulative_reward)),
        total=len(real_cumulative_reward),
        desc="將所有預測及真實累積報酬分類"):

        if idx in select_dict.keys():

            title = f'{select_dict[idx]}等分畫圖'
            ws = wb.create_sheet(title=title)

            ws.merge_cells('A1:B1')
            type_block(ws, '投資組合', 1, 1)
            type_block(ws, '公司數', 1, 3)
            type_block(ws, '964', 1, 4)
            type_block(ws, f'{select_dict[idx]}等分', 2, 1)
```

```

ws.merge_cells('A1:B1')
type_block(ws, '投資組合', 1, 1)
type_block(ws, '公司數', 1, 3)
type_block(ws, '964', 1, 4)
type_block(ws, f'{select_dict[idx]}等分', 2, 1)
type_block(ws, '時間', 2, 2)

storage(ws, [date], '填入時間', 2, 3, shape_vertical=True)
storage(ws, [element_id], '填入模型名稱', 3, 2)

type_block(ws, 'Real', 10, 2)
type_block(ws, '大盤', 11, 2)

for p_idx, pcr in enumerate(pcrs):
    storage(ws, [pcr], f'填入{element_id[p_idx]}的{select_dict[idx]}等分', 3+p_idx, 3, shape_vertical=True)
    fill_color(ws, 3, 3, 9, 136, color="FFDE9D9")

storage(ws, [rcr], f'填入真實累積報酬的{select_dict[idx]}等分', 10, 3, shape_vertical=True)
fill_color(ws, 10, 3, 10, 136, color="FFDAEEF3")
storage(ws, [market], f'填入大盤累積報酬', 11, 3, shape_vertical=True)
fill_color(ws, 11, 3, 11, 136, color="FFDAEEF3")

modify_block(ws)

```

wb.save(output)

- arr_cumulative 計算再投資累積報酬

```

def arr_cumulative(wb: Workbook, name: str, portfolio_data: List[List], market: List, inputs: Path) -> None:

    arr_data = []
    mk_data = []

    ws = ensure_sheet(wb, f'{name}再投資累積報酬')

    for participate_data in zip(*portfolio_data):
        cumprod = np.cumprod(np.array(participate_data)+1) -1
        arr_data.append(cumprod.tolist())

    type_block(ws, "TWII", 25, 1)
    type_block(ws, "大盤", 25, 2)
    copy_worksheet_range(wb["預測IC"], ws, 1, 24, 1, 136)
    storage(ws, arr_data, f"將再投資累積報酬儲存至工作表->{name}再累積投資報酬", 12, 3, shape_vertical=True)
    storage(ws, [market], f"將大盤再投資累積報酬儲存至工作表->{name}再累積投資報酬", 25, 3, shape_vertical=True)

    modify_block(ws)
    fill_color(ws, 25, 1, 25, 136, color="FFC6E0B4")

    wb.save(os.path.join(inputs, f'{name}.xlsx'))
    wb.close()

```

- data_store 儲存，美化表格

```
def data_store( file: Path, data: dict[str, Any], model: str) -> None:

    excel_data = openpyxl.load_workbook(file)

    ws_gt = ensure_sheet(excel_data, "真實IC")
    ws_ic = ensure_sheet(excel_data, "預測IC")
    ws_wr = ensure_sheet(excel_data, f"{MODEL_NAME[model]}勝率")
    ws_cr = ensure_sheet(excel_data, f"{MODEL_NAME[model]}累積報酬")
    ws_sp = ensure_sheet(excel_data, f"{MODEL_NAME[model]}夏普比率")
    ws_r2 = ensure_sheet(excel_data, f"{MODEL_NAME[model]}樣本外R2")

    IC = data["IC"] # 三因子IC數據
    ID = data["ID"] # 每個數據最後選出來的因子
    SV = data["select_values"] # 篩選出的因子的數據
    PD = data["portfolio_data"] # 所有投資組合數據
    PV = data["portfolio_value"] # 所有投資組合數據分類
    TC = data["total_count"] # 所有勝過大盤的數據總數
    WR = data["win_rate"] # 所有投資組合的勝率
    CR = data["cumulative_rewards"] # 所有投資組合的累積報酬率
    SP = data["sharpe_ratio"] # 夏普比率數據
    SM = data["sharpe_ratio_market"] # 夏普比率數據(大盤)
    PR = data["PR_RS"] # (預測-真實)^2
    RR = data["RR_RS"] # (真實)^2
    RS = data["RS"] # 整體樣本外R^2

    XR = data["max_records"] # 篩選的最大值的公司號
    NR = data["min_records"] # 篩選的最小值的公司號
    XC = data["max_company"] # 篩選的最大值的公司
    NC = data["min_company"] # 篩選的最小值的公司
```

```

# 整合篩選因子的數據並儲存
factors = [[i, ELEMENT_ID[i], sv] for i, sv in zip(ID, SV)]

# 儲存基本IC資訊
storage(ws_ic, IC, "儲存IC至工作表->預測IC", 3, 3)
storage(ws_ic, factors, "儲存篩選因子至工作表->預測IC", 6, 3)
storage(ws_ic, PD, "儲存投資組合至工作表->預測IC", 12, 3)

type_block(ws_ic, "低", 28, 2)
type_block(ws_ic, "公司名稱", 29, 2)
type_block(ws_ic, "高", 30, 2)
type_block(ws_ic, "公司名稱", 31, 2)

storage(ws_ic, [NR], '儲存最小值的公司號', 28, 3, shape_vertical=True)
storage(ws_ic, [NC], '儲存最小值的公司號', 29, 3, shape_vertical=True)
storage(ws_ic, [XR], '儲存最大值的公司號', 30, 3, shape_vertical=True)
storage(ws_ic, [XC], '儲存最大值的公司號', 31, 3, shape_vertical=True)

# 將以填入過的數據複製到其他工作表中，這樣就不需要在重複填上數據，不然會很亂
copy_worksheet_range(ws_ic, ws_wr, 1, 26, 1, 136)
copy_worksheet_range(ws_ic, ws_cr, 1, 26, 1, 136)

# 儲存勝率相關資訊
storage(ws_wr, PV, f"將投資組合評斷分數儲存至工作表->{model}勝率", 12, 3)
storage(ws_wr, [TC], f"將各等分數勝過大盤總數儲存至工作表->{model}勝率", 12, 137)
storage(ws_wr, [WR], f"將各等分勝率儲存至工作表->{model}勝率", 12, 138)
type_block(ws_wr, "sum", 11, 137)
type_block(ws_wr, "勝率", 11, 138)

# 儲存累積報酬相關資訊，因為這個與前面的形狀不同所以貼到工作表的位置需要做更正
storage(ws_cr, CR, f"將各等分數累積報酬儲存至工作表->{model}累積報酬", 12, 3, shape_vertical=True)

"""
儲存夏普比率

```

```

storage(ws_wr, [TC], f"將各等分數勝過大盤總數儲存至工作表->{model}勝率", 12, 137)
storage(ws_wr, [WR], f"將各等分勝率儲存至工作表->{model}勝率", 12, 138)
type_block(ws_wr, "sum", 11, 137)
type_block(ws_wr, "勝率", 11, 138)

# 儲存累積報酬相關資訊，因為這個與前面的形狀不同所以貼到工作表的位置需要做更正
storage(ws_cr, CR, f"將各等分數累積報酬儲存至工作表->{model}累積報酬", 12, 3, shape_vertical=True)

"""
儲存夏普比率
先複製預測IC的工作表資訊，在建立欄位名稱：平均值、標準差以及夏普比率
"""
copy_worksheet_range(ws_ic, ws_sp, 1, 26, 1, 136)
type_block(ws_sp, "平均值", 11, 137)
type_block(ws_sp, "標準差", 11, 138)
type_block(ws_sp, "夏普比率", 11, 139)
storage(ws_sp, SP, f"將夏普比率儲存至工作表->{model}夏普比率", 12, 137, shape_vertical=True)
storage(ws_sp, SM, f"將大盤的夏普比率儲存至工作表->{model}夏普比率", 26, 137, shape_vertical=True)

# 儲存R-square
copy_to_position(ws_gt, ws_r2, 1, 5, 1, 136, 7, 1) # 複製真實IC
copy_to_position(ws_ic, ws_r2, 1, 5, 1, 136, 1, 1) # 複製預測IC
copy_to_position(ws_ic, ws_r2, 3, 5, 2, 2, 13, 2)
copy_to_position(ws_ic, ws_r2, 3, 5, 2, 2, 17, 2)
storage(ws_r2, PR, f"將(預測-真實)^2 R-square儲存至工作表->{model}樣本外R2", 13, 3)
storage(ws_r2, RR, f"將(真實^2) R-square儲存至工作表->{model}樣本外R2", 17, 3)
fill_color(ws_r2, 13, 3, 15, 136, color="FFFE699")
fill_color(ws_r2, 17, 3, 19, 136, color="FFFE699")
type_block(ws_r2, "(預測-真實)^2", 13, 1)
type_block(ws_r2, "真實^2", 17, 1)
type_block(ws_r2, "R^2", 21, 1)
type_block(ws_r2, RS, 21, 2)

```

4. utils/data_store.py

```
# 將工作表的部分資訊貼到新的工作表，貼在跟舊的工作表一樣的位置
def copy_worksheet_range(
    source_ws: Worksheet,
    target_ws: Worksheet,
    min_row: int = 1,
    max_row: int = 26,
    min_col: int = 1,
    max_col: int = 136,
):
    for col in range(min_col, max_col + 1):
        col_letter = source_ws.cell(row=1, column=col).column_letter
        target_ws.column_dimensions[col_letter].width = source_ws.column_dimensions[
            col_letter
        ].width

    for row in range(min_row, max_row + 1):
        if row in source_ws.row_dimensions:
            target_ws.row_dimensions[row].height = source_ws.row_dimensions[row].height

    for row in source_ws.iter_rows(
        min_row=min_row, max_row=max_row, min_col=min_col, max_col=max_col
    ):
        for cell in row:
            new_cell = target_ws.cell(
                row=cell.row, column=cell.column, value=cell.value
            )
            if cell.has_style:
                new_cell.font = copy(cell.font)
                new_cell.border = copy(cell.border)
                new_cell.fill = copy(cell.fill)
                new_cell.number_format = copy(cell.number_format)
                new_cell.protection = copy(cell.protection)
                new_cell.alignment = copy(cell.alignment)

    for merged_cell_range in source_ws.merged_cells.ranges:
        minc, minr, maxc, maxr = merged_cell_range.bounds

        if minr >= min_row and maxr <= max_row and minc >= min_col and maxc <= max_col:
            target_ws.merge_cells(
                start_row=minr, start_column=minc, end_row=maxr, end_column=maxc
            )
```

```

def _collect_overlapping_merges(
    ws: Worksheet, top: int, left: int, bottom: int, right: int
) -> List[Tuple[int, int, int, int]]:

    overlaps = []
    for rng in ws.merged_cells.ranges:
        minc, minr, maxc, maxr = rng.bounds

        if not (maxr < top or minr > bottom or maxc < left or minc > right):
            overlaps.append((minr, minc, maxr, maxc))
    return overlaps

def copy_to_position(
    src_ws: Worksheet,
    tgt_ws: Worksheet,
    src_min_row: int,
    src_max_row: int,
    src_min_col: int,
    src_max_col: int,
    tgt_start_row: int,
    tgt_start_col: int,
):
    row_offset = tgt_start_row - src_min_row
    col_offset = tgt_start_col - src_min_col

    tgt_top = tgt_start_row
    tgt_left = tgt_start_col
    tgt_bottom = tgt_start_row + (src_max_row - src_min_row)
    tgt_right = tgt_start_col + (src_max_col - src_min_col)

    affected_merges = _collect_overlapping_merges(
        tgt_ws, tgt_top, tgt_left, tgt_bottom, tgt_right
    )
    for minr, minc, maxr, maxc in affected_merges:
        tgt_ws.unmerge_cells(
            start_row=minr, start_column=minc, end_row=maxr, end_column=maxc
        )

    for c in range(src_min_col, src_max_col + 1):
        src_letter = src_ws.cell(row=1, column=c).column_letter
        tgt_letter = tgt_ws.cell(row=1, column=c + col_offset).column_letter
        tgt_ws.column_dimensions[tgt_letter].width = src_ws.column_dimensions[
            src_letter
        ].width

    for r in range(src_min_row, src_max_row + 1):
        if r in src_ws.row_dimensions:

```

```

    for r in range(src_min_row, src_max_row + 1):
        if r in src_ws.row_dimensions:
            tgt_ws.row_dimensions[r + row_offset].height = src_ws.row_dimensions[
                r
            ].height

    for row in src_ws.iter_rows(
        min_row=src_min_row,
        max_row=src_max_row,
        min_col=src_min_col,
        max_col=src_max_col,
    ):
        for cell in row:
            new_r = cell.row + row_offset
            new_c = cell.column + col_offset
            tgt_cell = tgt_ws.cell(row=new_r, column=new_c)

            tgt_cell.value = cell.value

            if cell.has_style:
                tgt_cell.font = copy(cell.font)
                tgt_cell.border = copy(cell.border)
                tgt_cell.fill = copy(cell.fill)
                tgt_cell.number_format = copy(cell.number_format)
                tgt_cell.protection = copy(cell.protection)
                tgt_cell.alignment = copy(cell.alignment)

    for rng in src_ws.merged_cells.ranges:
        minc, minr, maxc, maxr = rng.bounds

        if (
            src_min_row <= minr <= src_max_row
            and src_min_col <= minc <= src_max_col
            and src_min_row <= maxr <= src_max_row
            and src_min_col <= maxc <= src_max_col
        ):
            tgt_ws.merge_cells(
                start_row=minr + row_offset,
                start_column=minc + col_offset,
                end_row=maxr + row_offset,
                end_column=maxc + col_offset,
            )

```

```

def storage(
    ws: Worksheet,
    results: List[List],
    work_name: str,
    row_s: int,
    col_s: int,
    shape_vertical: bool = False,
):
    if not shape_vertical:
        # for m_idx, elements in tqdm(
        #     enumerate(results), total=len(results), desc=f"{work_name}"
        # ):
        for m_idx, elements in enumerate(results):
            for idx, element in enumerate(elements):
                element = (
                    round(float(element), 8)
                    if isinstance(element, float)
                    else element
                )
                cell = ws.cell(row=row_s + idx, column=col_s + m_idx)
                cell.value = element
                cell.font = Font(name="Calibri", size=12, bold=True)
    else:
        # for m_idx, elements in tqdm(
        #     enumerate(results), total=len(results), desc=f"{work_name}"
        # ):
        for m_idx, elements in enumerate(results):
            for idx, element in enumerate(elements):
                element = (
                    round(float(element), 8)
                    if isinstance(element, float)
                    else element
                )
                cell = ws.cell(row=row_s + m_idx, column=col_s + idx)
                cell.value = element
                cell.font = Font(name="Calibri", size=12, bold=True)

```



```

def modify_block(ws: Worksheet) -> None:
    user_font = Font(
        name="Calibri",
        size=12,
        bold=True,
        italic=False,
        color="000000",
    )

    thick = Side(style="thick")
    thick_border = Border(top=thick, bottom=thick, left=thick, right=thick)
    center_align = Alignment(horizontal="center", vertical="center")

    for row in ws.iter_rows(
        min_row=ws.min_row,
        max_row=ws.max_row,
        min_col=ws.min_column,
        max_col=ws.max_column,
    ):
        for cell in row:
            if cell.value not in (None, ""):
                cell.border = thick_border
                new_font = copy(cell.font)
                for attr in (
                    "name",
                    "size",
                    "bold",
                    "italic",
                    "color",
                    "underline",
                    "strike",
                    "vertAlign",
                    "charset",
                    "scheme",
                ):
                    val = getattr(user_font, attr)
                    if val is not None:
                        setattr(new_font, attr, val)
                new_font.bold = True
                cell.font = new_font

            cell.alignment = center_align

```

```

def type_block(ws: Worksheet, value: Any, row: int, col: int) -> None:

    cell = ws.cell(row=row, column=col)
    cell.value = value
    cell.font = Font(name="Calibri", size=12, bold=True)
    cell.alignment = Alignment(horizontal="center", vertical="center")

def fill_color(
    ws: Worksheet,
    row_s: int,
    col_s: int,
    row_e: int,
    col_e: int,
    color: str = "FFFFE699",
) -> None:

    for r in range(row_s, row_e+1):
        for c in range(col_s, col_e+1):
            cell = ws.cell(row=r, column=c)
            cell.fill = PatternFill(fill_type="solid", fgColor=color)

```